Développement web et cloud

Cahier de conception

Un éditeur de documents partagé

Spécification et première conception

État de l'art, exigences, crédit, versions métier et MVP local

Auteur	MAYEGA BALOCK

Version	0.3 — Historique et clôture de l'édition
Date	25 octobre 2026
Modélisation	UML, sources PlantUML
Présentation	Noir et blanc

Objet et statut du cahier

Ce cahier prépare la conception d'une application web d'écriture collective et de diffusion de documents PDF sur AWS. Il présente les besoins, le modèle de crédit, le cadrage des versions métier et l'état rapporté du MVP local. Les garanties de quota, de clôture, de diffusion et de charge restent à démontrer.

Table des matières

Objet et statut du cahier **i**

1 État de l'art et étude de l'existant **1**

1.1 Problématique et méthode de comparaison 1

1.2 Mécanismes de collaboration 1

1.2.1 Transformation opérationnelle : OT 1

1.2.2 Types de données répliquées sans conflit : CRDT 1

1.2.3 Transport, présence et historique 2

1.3 Google Docs : fonctionnement publié et limites de connaissance 2

1.3.1 Fonctions pertinentes 2

1.3.2 Architecture du protocole décrite en 2010 2

1.4 Overleaf : collaboration et chaîne de production PDF 3

1.4.1 Mécanisme de collaboration et outils identifiés 3

1.4.2 Architecture sur site documentée 4

1.4.3 Persistance et limites de transposition 5

1.5 Synthèse comparative et enseignements 5

2 Analyse et spécification des exigences **6**

2.1 Contexte, parties prenantes et périmètre 6

2.2 Exigences utilisateur 7

2.3 Exigences fonctionnelles détaillées 7

2.3.1 Identification, documents et partage 7

2.3.2 Collaboration, présence et versions 8

2.3.3 Liens et distribution 9

2.3.4 Crédit quotidien et abus 10

2.3.5 Consultation de l'état 11

2.4 Règles métier et propriétés prioritaires 11

2.5 Exigences non fonctionnelles et contraintes 12

2.5.1 Exigences produit 12

2.5.2 Exigences organisationnelles et de réalisation 13

2.5.3 Exigences externes 15

2.5.4 Diagramme de contexte . . . 15
2.5.5 Diagramme de packages . . . 16
2.5.6 Diagramme de cas d'utilisation . . . 17
2.6 Scénarios ou cas d'utilisation structurés . . . 17
2.6.1 SC-01 — Créer et partager un document . . . 17
2.6.2 SC-02 — Écrire simultanément au même endroit . . . 18
2.6.3 SC-03 — Consulter puis restaurer une version . . . 18
2.6.4 SC-04 — Télécharger après une correction sans changer de lien . . . 19
2.6.5 SC-05 — Distribuer le dernier exemplaire sous accès concurrents . . . 19
2.6.6 SC-06 — Épuiser le crédit depuis plusieurs appareils . . . 19
2.6.7 SC-07 — Détecter un abus et observer le blocage . . . 20
2.6.8 SC-08 — Publier au pic et consulter l'état . . . 20
2.7 Vérification et traçabilité des contraintes . . . 21
2.8 Questions ouvertes et objectifs à valider . . . 22
2.9 Validation et gestion de la spécification . . . 23

3 Conception 25
3.1 Gestion du crédit, des débits et des écritures . . . 25
3.1.1 Objet et unité du crédit . . . 25
3.1.2 Coût proposé des opérations . . . 26
3.1.3 Acceptation, plafond et refus . . . 26
3.1.4 Journée, débit et abus . . . 26
3.1.5 Identité, sessions et appareils . . . 27
3.1.6 Cohérence du débit et décision à valider . . . 27
3.1.7 Diagramme de classes du crédit . . . 28
3.1.8 Diagramme d'activités du crédit . . . 29
3.1.9 Diagramme de séquence du crédit . . . 30
3.2 Historique, versionnage et branches . . . 30
3.2.1 Termes et organisation des documents . . . 30
3.2.2 Autorisations et édition en ligne . . . 31
3.2.3 Fermeture manuelle et passage quotidien . . . 32
3.2.4 Écritures refusées, invariants et décisions restantes . . . 33
3.2.5 Diagramme de classes des versions . . . 33
3.2.6 Diagramme d'activités de la clôture . . . 34
3.2.7 Diagramme de séquence de la clôture . . . 35
3.3 Création et partage des projets d'édition . . . 35
3.3.1 Règles et responsabilités . . . 35
3.3.2 Diagramme de classes du partage . . . 36

3.3.3 Diagramme d'activités de l'invitation 37
3.3.4 Diagramme de séquence du partage 38
3.3.5 Invariants et limites 38

4 Implémentation locale et premiers résultats 39
4.1 Choix technologiques et compromis de réalisation 39
4.2 Organisation et interfaces de cette étape 41
4.2.1 Points techniques à vérifier pour le crédit 42
4.3 Correspondance entre conception et code 42
4.4 Déploiement local et vérifications 43
4.5 Limites et infrastructure cible 43

Références 44

Chapitre 1

État de l'art et étude de l'existant

1.1 Problématique et méthode de comparaison

Un éditeur collaboratif doit concilier une interaction fluide et un contenu partagé cohérent. Le projet y ajoute des contraintes métier : crédit quotidien d'écriture, distribution d'un nombre exact d'exemplaires, contenu actualisé derrière un lien stable et maintien d'une réponse explicite pendant un pic de fréquentation .

L'état de l'art présente les mécanismes utilisables ; l'étude de l'existant observe leur emploi dans des produits. Google Docs est retenu pour l'édition généraliste dans le navigateur et Overleaf pour l'articulation entre collaboration, sources textuelles et production de PDF. Yjs est étudié comme bibliothèque alternative, sans lui attribuer un usage chez ces deux éditeurs.

La comparaison distingue trois niveaux de preuve : **fonction documentée**, **mécanisme technique publié** et **déploiement documenté**.

1.2 Mécanismes de collaboration

1.2.1 Transformation opérationnelle : OT

La transformation opérationnelle ajuste une opération par rapport aux changements intervenus depuis l'état du document connu par son auteur. Si un rédacteur insère du texte avant une zone qu'un autre modifie, les positions doivent être adaptées pour éviter de viser les mauvais caractères. Google l'illustre dans son explication de 2010 [4].

OT s'inscrit dans un protocole qui gère l'ordre des révisions, les opérations locales en attente et les acquittements. Le protocole historique de Google autorise l'application locale optimiste d'une saisie, puis sa réconciliation avec les opérations échangées avec le serveur [5]. L'affichage local d'une frappe ne suffit donc pas à définir son acceptation durable par le système.

1.2.2 Types de données répliquées sans conflit : CRDT

Une autre famille de solutions organise les données et les opérations pour permettre la fusion de modifications concurrentes. Yjs expose des types partagés et sépare le modèle de données du transport et des intégrations avec les éditeurs [14]. Sa documentation décrit des mises à jour commutatives, associatives et idempotentes ; les répliques convergent lorsqu'elles ont reçu toutes les mises à jour [15].

Cette propriété ne remplace pas le contrôle des autorisations, la persistance ni l'application d'un

crédit d'écriture partagé. Pour le projet, le choix OT ou CRDT devra être évalué avec les règles d'acceptation des modifications .

1.2.3 Transport, présence et historique

Un canal de transport livre les messages ; il ne résout pas à lui seul leurs conflits. Overleaf distingue ainsi OT pour réconcilier les modifications et WebSocket pour pousser les changements aux clients [9]. La présence et les positions d'écriture sont des informations complémentaires au contenu.

1.3 Google Docs : fonctionnement publié et limites de connaissance

1.3.1 Fonctions pertinentes

La documentation Google décrit le partage avec des rôles de lecteur, commentateur et éditeur [6], la consultation et la restauration des versions [7], ainsi que l'enregistrement automatique et le téléchargement des documents, dont la production d'un PDF [8]. Ces fonctions fournissent des références d'usage pour le partage, l'historique et la diffusion.

Une limite doit être prise en compte : l'aide Google annonce un maximum de 100 appareils ou onglets pouvant modifier simultanément un même fichier.

1.3.2 Architecture du protocole décrite en 2010

Dans la publication technique de Google, chaque client conserve ses opérations en attente et sa dernière révision synchronisée. Le serveur traite les opérations et enregistre les révisions. Il acquitte l'émetteur et communique les changements aux autres clients ; ceux-ci les réconcilient avec leur état local [5]. Le journal d'opérations et la transformation des positions sont décrits dans l'article consacré à la résolution des conflits [4].

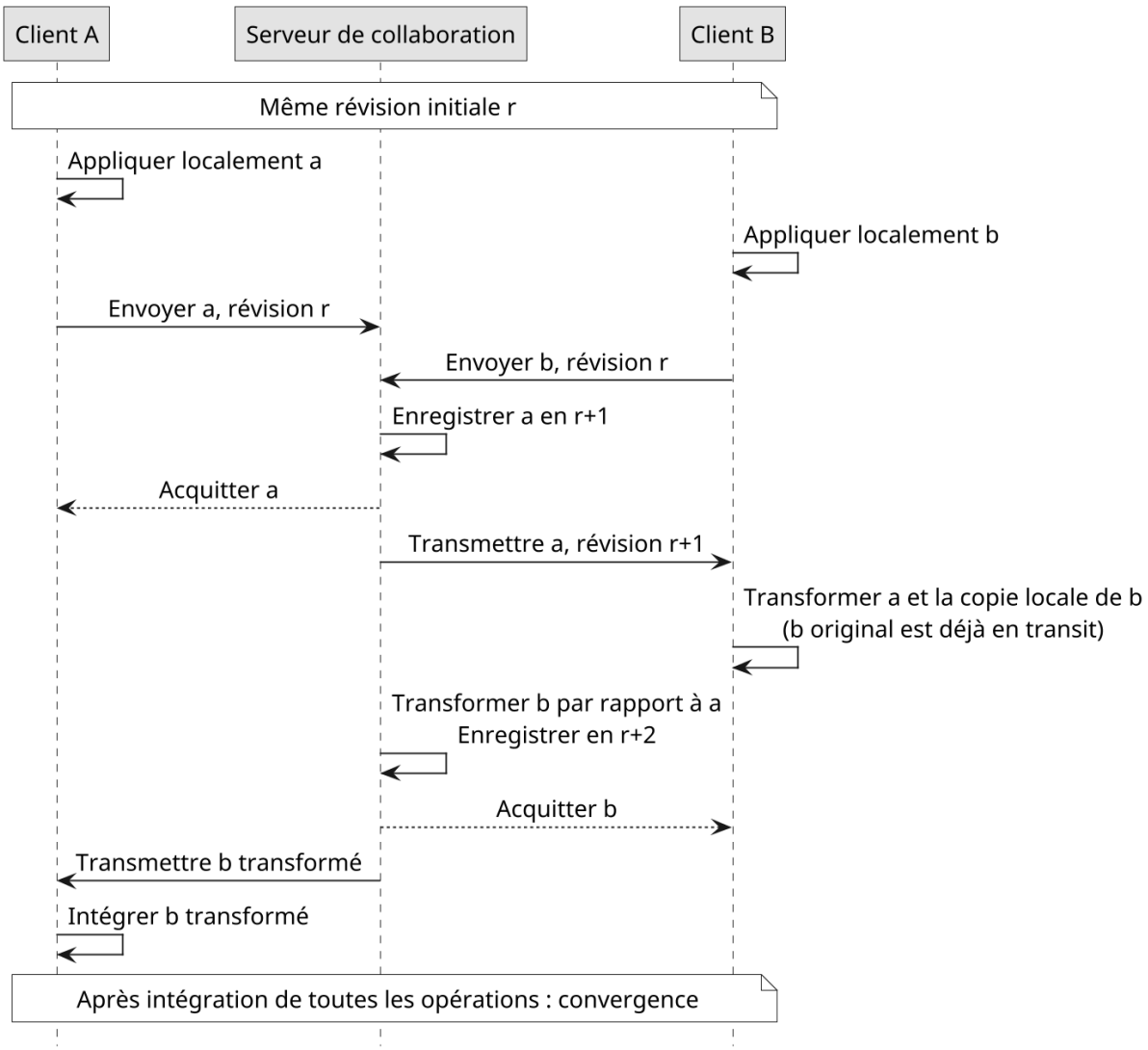


FIGURE 1.1 – Séquence UML simplifiée du protocole Google publié en 2010 ; illustration documentaire, sans prétention sur la pile actuelle.

1.4 Overleaf : collaboration et chaîne de production PDF

1.4.1 Mécanisme de collaboration et outils identifiés

Overleaf documente explicitement l'emploi d'OT : les changements sont transmis au serveur, qui les rebase pour faire converger les clients. WebSocket assure la notification des changements [9].

Le dépôt officiel déclare un moteur Node.js, de l'outillage TypeScript, ainsi que des dépendances ou résolutions associées à React et CodeMirror [10]. Ce manifeste identifie des technologies du code publié ; il ne démontre pas à lui seul leur usage dans chaque écran ni la topologie du service commercial. OT désigne ici le mécanisme de collaboration, tandis que les bibliothèques d'interface répondent à une autre responsabilité.//quel responsabilite

1.4.2 Architecture sur site documentée

La documentation des éditions sur site distingue les services `web`, `real-time`, `document-updater`, `docstore`, `filestore` et `clsi`. Leurs rôles couvrent les requêtes, les sockets, les modifications, les documents, les fichiers binaires et la compilation. Redis intervient dans les mises à jour actives et MongoDB dans le stockage des documents ; le Toolkit peut fournir des conteneurs pour ces dépendances [11].

L'infrastructure Server Pro documente Docker et un accès web servi par Nginx. La compilation isolée de Server Pro utilise des conteneurs ; la documentation distingue ce fonctionnement des sous-processus utilisés par Server CE [12, 11].

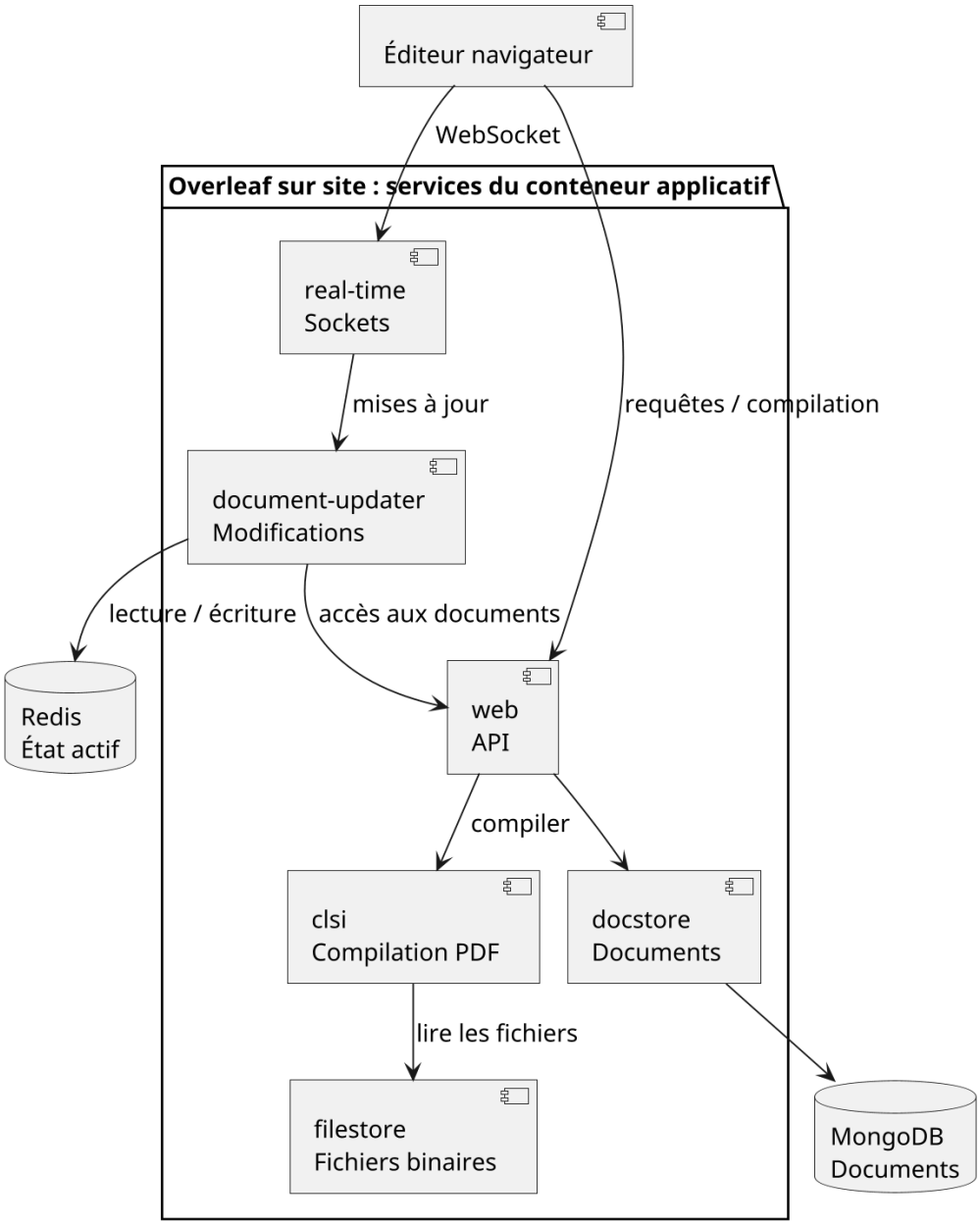


FIGURE 1.2 – Diagramme UML de composants : vue simplifiée des éditions Overleaf sur site, d'après leur documentation. Les dépendances expriment les échanges principaux.

1.4.3 Persistance et limites de transposition

La documentation de sauvegarde indique que Redis contient notamment des mises à jour en attente avant leur transfert vers MongoDB et recommande la persistance AOF [13]. Ce point attire l'attention sur la distinction entre données en mémoire et données récupérables : une politique de cache ne constitue pas, à elle seule, une preuve de conservation des écritures acceptées.

Overleaf montre également l'intérêt de séparer édition et génération PDF. Le sujet demandé porte toutefois sur du texte simple : il n'impose ni la saisie LaTeX ni un moteur de compilation TeX pour l'application. LaTeX est ici le format du cahier de conception.

1.5 Synthèse comparative et enseignements

Critère	Google Docs	Overleaf	Projet étudié
Collaboration	OT dans les articles historiques de 2010	OT et WebSocket documentés	Conservation et convergence obligatoires ; mécanisme à choisir
Historique	Consultation et restauration documentées	Historique présent dans le code et les services publiés ; modalités non évaluées ici	Consultation et restauration obligatoires
Production PDF	Téléchargement / impression PDF documentés	Service de compilation identifié sur site	PDF actuel, complet et cohérent
Infrastructure connue	Pile actuelle complète non établie	Docker, Nginx, Redis, MongoDB et services sur site documentés	AWS, une région et S3 imposés
Plafonds métier	Garanties spécifiques non établies par les sources étudiées	Garanties spécifiques non établies par les sources étudiées	Téléchargements exacts et crédit quotidien imposés
Validation	Aucune extrapolation de charge retenue	Aucune extrapolation de charge retenue	Tests et preuves propres au projet requis

TABLE 1.1 – Comparaison limitée aux faits documentés et aux besoins du sujet.

L'étude fait ressortir quatre responsabilités à examiner lors de la conception : synchroniser les modifications, définir leur acceptation, produire une version PDF cohérente et contrôler les consommations métier. La convergence d'un éditeur ne garantit pas l'exactitude d'un compteur de téléchargements. De même, un fichier disponible dans S3 ne suffit pas à établir une limitation du nombre de distributions.

Chapitre 2

Analyse et spécification des exigences

2.1 Contexte, parties prenantes et périmètre

L'association prépare collectivement un journal numérique. Plusieurs centaines de rédacteurs écrivent et corrigent les textes. À une heure annoncée, des visiteurs utilisent un lien public limité à 500 exemplaires ; les corrections se poursuivent pendant cette diffusion [1, p. 1].

Partie prenante	Type	Interaction
Rédacteur	Utilisateur final	Créer, partager, écrire, consulter/restaurer des versions et créer des liens.
Créateur d'un document ou d'un lien	Rôle d'un rédacteur	Donner accès au document ; choisir le tirage du lien ; recevoir l'avis d'épuisement.
Visiteur anonyme	Utilisateur final	Obtenir le PDF actuel et connaître l'état de sa demande sans déclarer d'identité.
Association étudiante	Propriétaire métier	Publier le journal et consulter les documents ouverts, les liens et les blocages.
Porteur du projet	Développeur / gestionnaire	Déployer, tester, expliquer les choix, supprimer les ressources et justifier le coût.
Jury	Partie prenante externe	Évaluer la conception et les preuves ; écrire des tests à partir de l'API.
AWS	Fournisseur externe	Héberger les ressources ; conserver les fichiers distribués dans S3.
Relais d'e-mail	Système externe	Remettre les codes de vérification et les notifications ; fournisseur et mode d'intégration non choisis.

Une même personne peut exercer plusieurs rôles. Les droits de consultation de la supervision restent à préciser ; le rôle de l'association n'impose pas à lui seul un type de compte administrateur.

Inclus : texte simple, partage, collaboration et présence, versions, liens limités, export PDF actuel, crédit quotidien, blocage temporaire, notification et supervision. **Non demandés explicitement :** traitement et presentation de texte riche.

2.2 Exigences utilisateur

ID	Besoin utilisateur
EU-01	Un rédacteur doit pouvoir créer un document textuel et le partager avec d'autres rédacteurs.
EU-02	Plusieurs rédacteurs doivent pouvoir travailler simultanément sur le même texte avec une vue partagée des changements, des présences et des positions d'écriture.
EU-03	Un rédacteur autorisé doit pouvoir consulter les états précédents d'un document ; son propriétaire doit pouvoir en restaurer un.
EU-04	Un rédacteur doit pouvoir diffuser le PDF actuel d'un document par un même lien, dans la limite du nombre de téléchargements qu'il a choisi.
EU-05	Chaque rédacteur doit être soumis au même crédit quotidien sur tous ses appareils et les tentatives répétées de contournement doivent entraîner un blocage temporaire.
EU-06	Un visiteur doit pouvoir utiliser un lien sans s'identifier et connaître l'issue de sa demande, y compris pendant le pic de publication.
EU-07	L'association doit pouvoir consulter les documents ouverts, les liens actifs et leurs capacités restantes, ainsi que les personnes bloquées.
EU-08	Le créateur d'un lien doit être averti par e-mail lorsque celui-ci est épuisé.

Justifications métier. EU-01 et EU-02 permettent la rédaction commune ; EU-03 permet de revenir sur l'évolution du texte ; EU-04 et EU-06 organisent la publication publique et ses corrections ; EU-05 applique les limites du service ; EU-07 rend la situation consultable ; EU-08 informe le responsable de la diffusion.

2.3 Exigences fonctionnelles détaillées

Chaque énoncé exprime une fonction. La source et le besoin parent assurent sa traçabilité et portent sa justification métier. Les critères constituent un protocole de vérification à préparer, et non des résultats déjà obtenus. Les mentions « à préciser » renvoient au registre des questions de la section 2.8.

2.3.1 Identification, documents et partage

EF-01 — Présentation du rédacteur

Le système doit permettre à un rédacteur de se présenter au moyen d'un nom et d'une adresse e-mail.

Vérification : Le nom et l'adresse sont saisis ; un code envoyé par e-mail vérifie la possession de l'adresse avant l'activation d'une identité métier stable. Ce code est une précision décidée par le porteur, non une obligation formulée dans le sujet..

EF-02 — Création d'un document

Le système doit permettre à un rédacteur de créer un document textuel.

Vérification : Après une création réussie, un document identifiable peut être ouvert et édité..

EF-03 — Partage du document

Le système doit permettre au créateur d'un document d'accorder son accès à un autre rédacteur.

Vérification : Le propriétaire attribue la lecture ou l'écriture sur une branche ; l'écriture ne vaut que pour sa version active ouverte. Le mode de désignation du destinataire reste à préciser..

EF-04 — Ouverture d'un document accessible

Le système doit permettre à un rédacteur disposant d'un accès d'ouvrir le document correspondant.

Vérification : Le document créé ou partagé est accessible au rédacteur concerné. C'est une décomposition opérationnelle du partage, pas une politique de rôles supplémentaire..

2.3.2 Collaboration, présence et versions

EF-05 — Écriture simultanée

Le système doit permettre à plusieurs rédacteurs de soumettre simultanément des modifications d'un même document.

Vérification : Plusieurs sessions autorisées peuvent modifier la version active ouverte d'une branche, y compris à la même position ; une version figée refuse l'écriture..

EF-06 — Diffusion des modifications

Le système doit communiquer aux autres participants les modifications acceptées du document.

Vérification : Une modification acceptée dans une session devient visible dans les autres sessions participantes. Délai : voir ENF « presque aussitôt », à chiffrer..

EF-07 — Affichage des présences

Le système doit afficher les rédacteurs présents dans le document.

Vérification : Des sessions actives distinctes apparaissent dans la vue de présence. Définition de présence et délai après déconnexion à préciser..

EF-08 — Affichage des positions

Le système doit afficher la position d'écriture des autres rédacteurs présents.

Vérification : Le déplacement de la position d'écriture dans une session est reflété pour les autres ; représentation exacte à choisir..

EF-09 — Conservation des versions

Le système doit conserver des versions précédentes du document.

Vérification : Une branche garde ses versions métier figées et leurs révisions techniques consultables ; la politique de rétention reste à préciser. Une frappe ne crée pas à elle seule une nouvelle version métier..

EF-10 — Consultation d'une version

Le système doit permettre à un rédacteur de consulter une version précédente du document.

Vérification : Le droit de lecture de la branche donne accès à ses versions et révisions historiques, sans confondre leur contenu avec celui de la version active..

EF-11 — Restauration d'une version

Le système doit permettre à un rédacteur de restaurer une version précédente du document.

Vérification : Le propriétaire choisit une version et une révision source ; la restauration crée une nouvelle branche et sa première version éditable, sans modifier la source ni recopier automatiquement ses permissions..

2.3.3 Liens et distribution

EF-12 — Création d'un lien

Le système doit permettre à un rédacteur de créer un lien de téléchargement pour un document.

Vérification : La création réussie fournit un lien utilisable pour le document désigné..

EF-13 — Choix du nombre de téléchargements

Le système doit permettre au créateur de choisir le nombre de téléchargements autorisés du lien.

Vérification : Le nombre choisi est associé au lien et vérifiable ; règles sur zéro, nombres négatifs et plafond à préciser..

EF-14 — Fourniture du PDF

Le système doit fournir un fichier au format PDF lors d'un téléchargement autorisé.

Vérification : Le fichier obtenu est un PDF lisible contenant le document..

EF-15 — Sélection du contenu actuel

Le système doit déterminer le contenu téléchargé à partir de l'état actuel du document lors de l'utilisation du lien.

Vérification : Après modification du document, le même lien restitue la correction lors d'une utilisation ultérieure. Instant précis de référence à valider..

EF-16 — Refus au-delà du plafond

Le système doit refuser un téléchargement supplémentaire lorsque le nombre autorisé est épuisé.

Vérification : Pour un lien de capacité N, des appels concurrents ne produisent jamais plus de N distributions comptabilisées. Définition de la comptabilisation à valider..

EF-17 — Affichage de l'épuisement

Le système doit indiquer au visiteur que le lien est épuisé lorsqu'aucun téléchargement n'est encore autorisé.

Vérification : Une demande après épuisement reçoit un résultat explicite « épuisé »..

EF-18 — Téléchargement anonyme

Le système doit permettre à un visiteur de télécharger sans fournir d'identité.

Vérification : Un visiteur ne disposant ni de nom saisi ni d'adresse e-mail peut obtenir un téléchargement autorisé..

EF-19 — Affichage de la disponibilité

Le système doit indiquer au visiteur lorsqu'il peut télécharger immédiatement.

Vérification : Un résultat de demande autorisée indique clairement la disponibilité immédiate du téléchargement..

EF-20 — Affichage de l'attente

Le système doit indiquer au visiteur lorsqu'il doit patienter.

Vérification : Si le système diffère le traitement, la demande reçoit un état explicite d'attente. Cela n'impose pas un service de file particulier..

EF-21 — Information des visiteurs présents

Le système doit informer de l'épuisement les visiteurs encore présents après le dernier exemplaire distribué.

Vérification : Un visiteur déjà en attente apprend l'état final sans rester indéfiniment en attente ; délai maximal à chiffrer..

EF-22 — Notification au créateur

Le système doit envoyer un e-mail au créateur du lien lorsque le dernier téléchargement autorisé est consommé.

Vérification : Le passage de capacité restante 1 à 0 déclenche l'avis destiné à l'adresse du créateur. Délai, reprise sur échec et doublons à spécifier..

2.3.4 Crédit quotidien et abus

EF-23 — Attribution du crédit quotidien

Le système doit attribuer à chaque rédacteur un crédit d'écriture de 10 000 caractères par jour.

Vérification : Au début d'une période quotidienne, le crédit accordé est de 10 000 caractères selon l'unité de comptage retenue..

EF-24 — Décompte de l'écriture

Le système doit prendre en compte l'écriture acceptée dans le crédit du rédacteur concerné.

Vérification : Une écriture acceptée réduit le crédit conformément à la règle de comptage ; les réinsertions volontaires sont contrôlées. Le coût de la copie initiale d'une nouvelle version reste à valider..

EF-25 — Refus du dépassement de crédit

Le système doit refuser l'écriture excédant le crédit restant du rédacteur.

Vérification : Un rédacteur ne peut faire accepter un total supérieur à 10 000 caractères sur la période ; rejet complet ou partiel d'une opération à préciser..

EF-26 — Crédit commun aux appareils

Le système doit appliquer un crédit commun aux appareils utilisés simultanément par un même rédacteur.

Vérification : Avec deux appareils consommant le reliquat en même temps, le total accepté demeure inférieur ou égal au crédit de la période.

EF-27 — Recharge quotidienne

Le système doit recharger le crédit d'écriture chaque jour.

Vérification : Après changement de période quotidienne, le rédacteur retrouve le crédit prévu ; fuseau et instant de recharge à fixer..

EF-28 — Détection des abus

Le système doit repérer les tentatives répétées d'outrepassement des limites.

Vérification : Une séquence dépassant une règle de détection validée est signalée comme abusive. Seuil et fenêtre temporelle à définir.

EF-29 — Blocage temporaire

Le système doit bloquer temporairement une personne repérée pour ces tentatives.

Vérification : Après détection, les actions couvertes par le blocage sont refusées pendant la durée retenue ; identité ciblée, périmètre et durée à préciser.

2.3.5 Consultation de l'état

EF-30 — Consultation des documents ouverts

Le système doit permettre à l'association de consulter les documents ouverts.

Vérification : La vue d'état restitue les documents répondant à la définition validée d'« ouvert »..

EF-31 — Consultation des liens actifs

Le système doit permettre à l'association de consulter les liens actifs.

Vérification : La vue d'état identifie les liens répondant à la définition validée d'« actif »..

EF-32 — Consultation des téléchargements restants

Le système doit présenter le nombre de téléchargements restants de chaque lien actif.

Vérification : Le reste présenté correspond au nombre autorisé diminué des consommations comptabilisées ; fraîcheur à préciser..

EF-33 — Consultation des personnes bloquées (non prioritaire)

Le système doit permettre à l'association de consulter les personnes actuellement bloquées.

Vérification : Une personne sous blocage apparaît dans la vue, puis cesse d'y être considérée bloquée à l'expiration..

2.4 Règles métier et propriétés prioritaires

RM-01 — Tirage d'un lien. Pour un lien L, la capacité initiale N_L est choisie à sa création. Le nombre de consommations comptabilisées $D_L(t)$ doit satisfaire $0 \leq D_L(t) \leq N_L$ à tout instant. Lorsqu'au moins N_L demandes admissibles aboutissent, le système doit permettre les N_L consommations prévues.

RM-02 — Modifications acceptées. Toute modification acceptée doit être intégrée selon la sémantique d'édition retenue. Après réception et traitement de toutes les opérations, les participants doivent voir le même document. L'égalité des copies ne prouve pas seule l'absence de perte : elles pourraient avoir perdu la même modification. Une suppression ultérieure légitime ne signifie pas non plus qu'une insertion antérieure n'a jamais été intégrée. Source : C2 ; EF-05, EF-06 ; P0.

RM-03 — Contenu actuel. Le PDF doit correspondre à un état complet et cohérent du document à l'instant de référence de l'utilisation du lien. Il ne doit pas assembler des fragments de versions incompatibles. Le même lien doit refléter les corrections intervenues depuis sa création.

RM-04 — Crédit quotidien. Pour un rédacteur u et une journée j, le total d'écriture facturée et acceptée $E(u, j)$ doit satisfaire $E(u, j) \leq 10\,000$, tous appareils confondus. L'unité de caractère,

les opérations facturées, la journée et l'identité commune doivent être définies avant les tests définitifs.

2.5 Exigences non fonctionnelles et contraintes

La classification reprend les exigences **produit**, **organisationnelles** et **externes** du cours [2, diapositives 9–14]. Les invariants RM-01 à RM-04 sont également reliés aux qualités de fiabilité et de cohérence, sans devenir des règles différentes.

2.5.1 Exigences produit

ENF-01 — Exactitude du tirage

Le système doit préserver RM-01 sous accès concurrents.

Vérification : Soumettre davantage de demandes concurrentes que la capacité, notamment N=1 puis N=500 ; constater zéro dépassement et N consommations possibles lorsque les demandes admissibles aboutissent..

ENF-02 — Conservation des modifications

Le système doit préserver les modifications acceptées définies par RM-02, y compris lorsque les rédacteurs écrivent au même endroit.

Vérification : Journaliser les opérations acceptées et vérifier leur intégration avec un oracle adapté au modèle retenu ; inclure insertions, suppressions et remplacements concurrents..

ENF-03 — Convergence des participants

Les participants doivent finir par voir le même document après traitement des modifications acceptées.

Vérification : Après arrêt des nouvelles éditions et traitement des messages, comparer toutes les copies. Zéro divergence dans les essais ; délai maximal d'observation à fixer (Q-11)..

ENF-04 — Cohérence et fraîcheur du PDF

Le système doit préserver RM-03 pendant les modifications et les téléchargements simultanés.

Vérification : Comparer le texte extrait du PDF à la révision de référence acquise à T1 et vérifier que le fichier est lisible et complet. Q-03 détermine T1..

ENF-05 — Intégrité du crédit partagé

Le système doit préserver RM-04 même lorsqu'un rédacteur écrit depuis plusieurs appareils simultanément.

Vérification : Faire consommer simultanément un faible reliquat sur deux appareils ; vérifier le total par identité et journée et les refus au-delà de la limite..

ENF-06 — Rapidité de collaboration

Les modifications des autres participants doivent devenir visibles presque aussitôt.

Vérification : Mesurer la latence de diffusion et publier les percentiles et le maximum ; seuil absent du sujet, à définir en Q-11..

ENF-07 — Capacité pendant la publication

Le système doit supporter 15 000 visiteurs, dont 80 % dans la première minute, pendant que 300 rédacteurs continuent d'écrire.

Vérification : Exécuter le profil V-05 en conservant toutes les réponses et opérations. Les 12 000 arrivées de première minute valent 200/s en moyenne ; ni les rafales ni le débit total de requêtes ne sont définis par ce calcul..

ENF-08 — Réponse explicite sous charge

Chaque visiteur doit obtenir pendant le pic une réponse indiquant téléchargement immédiat, attente ou épuisement.

Vérification : Réconcilier chaque demande avec son résultat métier. Une erreur technique inexpliquée ou une attente sans retour ne valide pas le besoin ; délais à fixer en Q-11..

ENF-09 — Propagation de l'épuisement

Les visiteurs encore présents doivent apprendre rapidement l'épuisement du lien.

Vérification : Mesurer du dernier téléchargement comptabilisé à l'affichage chez les visiteurs actifs ; seuil à définir en Q-11..

ENF-10 — Fin des traitements devenus inutiles

Le système doit cesser de proposer la distribution au-delà de la capacité d'un lien épuisé.

Vérification : Vérifier l'absence de nouvelle autorisation et examiner les traitements déclenchés après épuisement. La portée des travaux à arrêter reste à préciser (Q-14) ; l'édition et les autres liens doivent rester utilisables..

ENF-11 — Protection des clés AWS

Aucune clé d'accès AWS ne doit apparaître dans le code ou dans le dépôt.

Vérification : Inspecter les fichiers, l'historique Git et les artefacts produits avec un contrôle de secrets et une revue de configuration ; conserver le résultat sans reproduire de secret..

ENF-12 — Permissions minimales

Chaque composant doit disposer seulement des droits nécessaires à ses fonctions.

Vérification : Relier chaque permission à une action et à une ressource nécessaires ; examiner les permissions effectives et tester le refus d'opérations non requises..

2.5.2 Exigences organisationnelles et de réalisation

Ces contraintes sont imposées par les règles du projet. Le fait qu'AWS soit un fournisseur externe ne les classe pas automatiquement parmi les exigences externes au sens de Sommerville.

ENF-13 — Environnement AWS

L'application doit être déployée sur le propre compte AWS du porteur du projet, en plan gratuit.

Vérification : Inventorier les ressources et documenter le compte et son plan sans exposer d'identifiants sensibles. Le sujet ne fixe pas de budget chiffré..

ENF-14 — Région unique

Toutes les ressources du projet doivent être dans une seule région AWS.

Vérification : Produire l'inventaire régional ; examiner toute proposition de service global avant de l'adopter, afin de clarifier sa compatibilité avec cette règle..

ENF-15 — Stockage des exports

Les fichiers proposés au téléchargement doivent être conservés dans S3.

Vérification : Relier un PDF distribué à l'objet S3 correspondant ; vérifier configuration et contenu. Cette contrainte n'impose pas S3 pour toutes les données applicatives..

ENF-16 — Travail personnel

Le projet doit rester strictement personnel.

Vérification : Le porteur doit pouvoir expliquer les choix et l'ensemble rendu..

ENF-17 — Bonnes pratiques

La réalisation doit appliquer les bonnes pratiques vues en cours.

Vérification : Établir la liste des pratiques depuis les supports pertinents et la relier aux preuves ; la liste exhaustive ne peut pas être déduite du seul cours de spécification fourni..

ENF-18 — Reproductibilité du rendu

Le dépôt Git doit contenir le code, la description d'infrastructure, les tests et un README expliquant comment déployer, tester et supprimer l'ensemble.

Vérification : Un lecteur suit le README depuis les prérequis déclarés ; contrôler la création puis la suppression des ressources. Aucun outil d'infrastructure particulier n'est imposé..

ENF-19 — Dossier de conception

Le rendu doit comprendre un dossier de conception.

Vérification : Vérifier la présence du dossier et la justification des choix. Le présent cahier en constitue la première partie..

ENF-20 — API vérifiable par le jury

La description de l'API doit permettre au jury d'écrire son propre test.

Vérification : Documenter entrées, sorties, erreurs, états et sémantiques de consommation et d'acceptation ; effectuer une revue de reproductibilité indépendante..

ENF-21 — Coût justifié

Le rendu doit fournir une preuve de ce que le projet a coûté.

Vérification : Présenter un relevé daté avec période et périmètre ; distinguer coûts, crédits et estimations. Le plan gratuit ne constitue pas à lui seul une preuve de coût nul..

ENF-22 — Format du cahier

Le cahier doit être maintenu en LaTeX compilable, en noir et blanc, avec des diagrammes UML dont les sources sont en PlantUML.

Vérification : Compiler les sources, vérifier la présentation et régénérer les diagrammes à partir des fichiers .puml..

ENF-23 — Délai de réalisation

La réalisation doit tenir du vendredi soir au lundi matin.

Vérification : Conserver un planning et des traces datées ; les dates civiles exactes ne sont pas fournies..

ENF-24 — Rapport de test

Le rendu doit comprendre un rapport de test avec méthode, objectifs, résultats bruts, analyse et preuves des deux contraintes prioritaires.

Vérification : Relier les résultats aux scénarios, à la configuration et à la version testée ; distinguer résultats obtenus et limites. Aucun test de l'application n'est déclaré exécuté dans ce cahier..

2.5.3 Exigences externes

Le sujet ne nomme aucune loi, certification ou contrainte réglementaire précise. Aucune conformité juridique particulière n'est donc déclarée acquise dans ce registre. L'utilisation de noms et d'adresses e-mail nécessite d'identifier ultérieurement les obligations effectivement applicables ; les exigences correspondantes devront être sourcées et ajoutées sans inventer une norme imposée par le sujet.

2.5.4 Diagramme de contexte

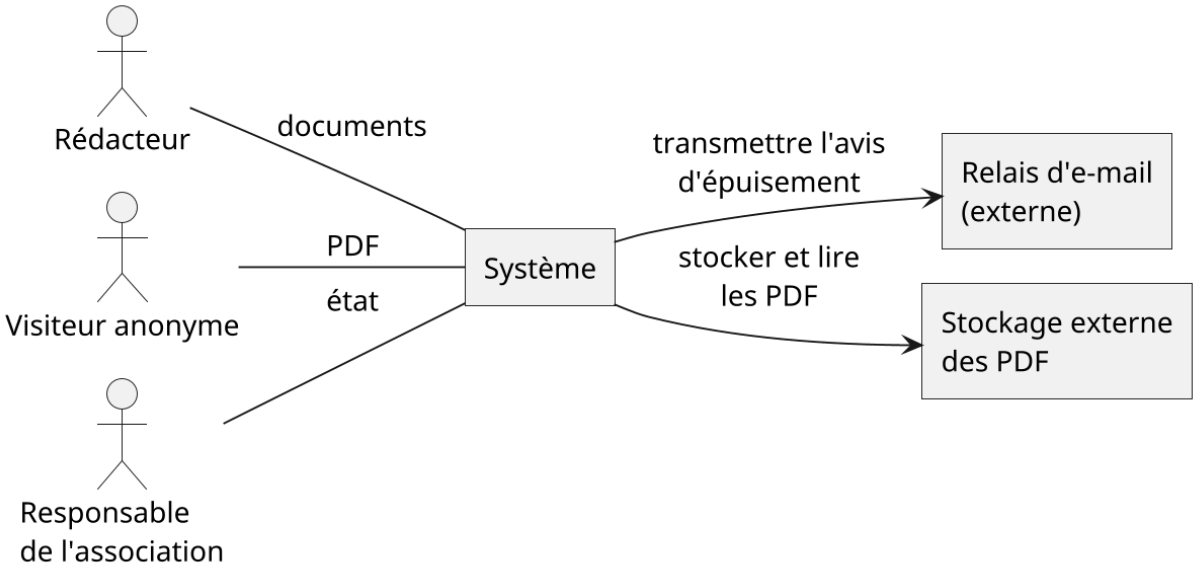


FIGURE 2.1 – Diagramme de contexte UML : le système, ses acteurs humains et ses dépendances externes.

2.5.5 Diagramme de packages

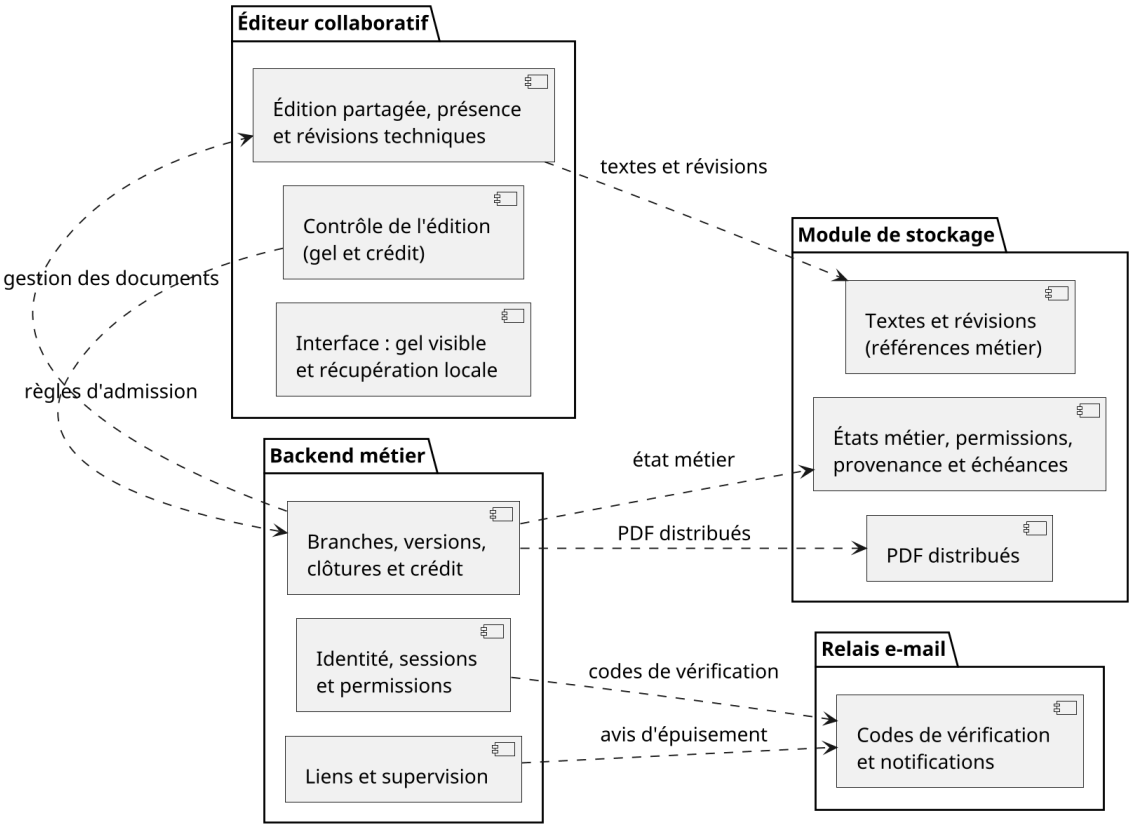


FIGURE 2.2 – Packages UML : édition, règles métier et conservation des données.

L'éditeur collaboratif gère le texte partagé, la présence et les révisions techniques. Son interface doit signaler le gel et permettre la récupération du texte local refusé ; l'extension métier doit participer au contrôle de l'édition. Le backend gère l'identité stable, les sessions, les droits de branche, les versions métier, les échéances de clôture, le crédit, les liens et la supervision. Le relais e-mail remet les codes de vérification et les notifications. Le module de stockage conserve les états, les permissions, la provenance, les échéances, les références des documents et révisions, ainsi que les PDF distribués. Ces packages sont des frontières logiques ; le diagramme ne promet ni transaction commune ni service déployé par composant.

Le diagramme suivant présente les fonctions accessibles aux acteurs.

2.5.6 Diagramme de cas d'utilisation

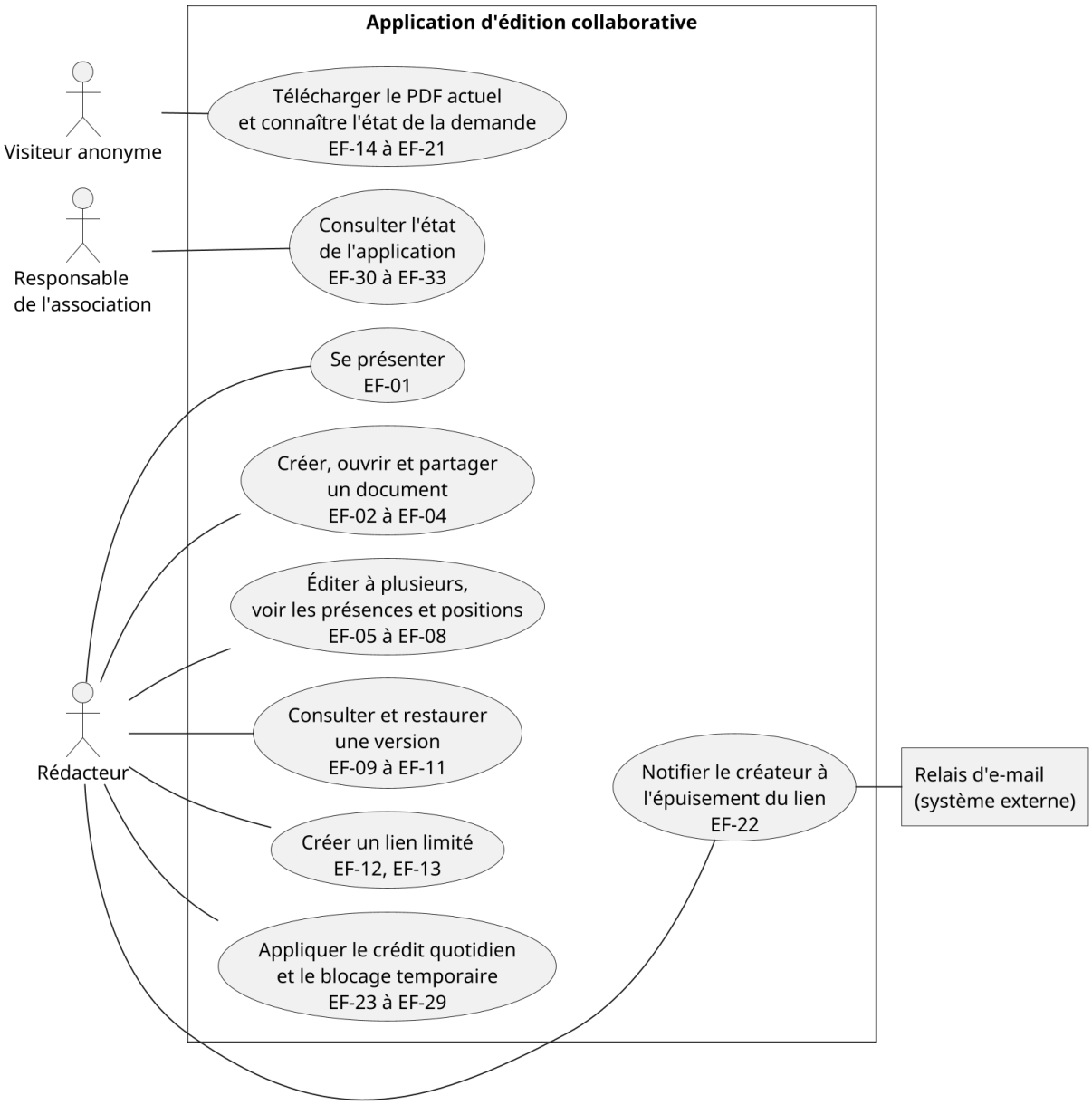


FIGURE 2.3 – Diagramme UML global des cas d'utilisation et des acteurs du projet.

2.6 Scénarios ou cas d'utilisation structurés

2.6.1 SC-01 — Créer et partager un document

Acteur principal	Rédacteur qui crée le document.
Acteurs secondaires	Second rédacteur, auquel le créateur partage le document.
Précondition	Les deux rédacteurs sont identifiés par leur nom et leur adresse e-mail ; le document à créer n'existe pas encore.

Scénario	1. Le rédacteur principal crée un document textuel. 2. Il désigne le second rédacteur et lui accorde l'accès au document. 3. Le second rédacteur ouvre le document partagé. Variantes : l'identité du destinataire est ambiguë ou inexistante, les données saisies sont invalides, ou la création échoue. Le comportement attendu dans ces cas reste à préciser. En parallèle, d'autres documents peuvent être créés ou ouverts sans effet de bord sur celui-ci.
Post-condition	Le document existe et les deux rédacteurs concernés peuvent y accéder.

2.6.2 SC-02 — Écrire simultanément au même endroit

Acteur principal	Rédacteurs qui éditent simultanément le même document.
Acteurs secondaires	Autres participants au document, qui reçoivent les modifications et les positions d'écriture.
Précondition	La version active de la branche est OUVERTE aux rédacteurs ; leur crédit est disponible et aucun n'est bloqué ; au moins deux rédacteurs éditent la même zone.
Scénario	1. Les rédacteurs soumettent leurs modifications concurrentes. 2. Le système détermine lesquelles sont acceptées et diffuse les changements ainsi que les positions. 3. Après l'arrêt des éditions et le traitement des échanges, les participants voient le même document. Variantes : crédit épuisé, déconnexion ou répétition d'une requête après perte de réponse ; les règles d'acceptation et de reprise restent à valider. En parallèle, d'autres rédacteurs peuvent éditer, créer des liens et télécharger le document.
Post-condition	Les modifications acceptées sont intégrées, les vues convergent et les crédits restent conformes.

Traçabilité : EU-02, EU-05 ; EF-05 à EF-08, EF-23 à EF-26 ; RM-02, RM-04.

2.6.3 SC-03 — Consulter puis restaurer une version

Acteur principal	Propriétaire de la branche source.
Acteurs secondaires	Rédacteurs autorisés sur la branche source.
Précondition	Le propriétaire peut consulter la version et la révision choisies dans la branche source.
Scénario	1. Le propriétaire choisit une version et une révision antérieures et consulte leur contenu. 2. Il demande la restauration ; le système crée une branche distincte et sa première version éditable à partir de ce contenu. 3. Le propriétaire attribue les accès à la nouvelle branche. Variantes : la source est absente ou inaccessible ; la création échoue et ne doit pas altérer la branche source. En parallèle, les rédacteurs peuvent continuer sur la branche source sans modifier la nouvelle.

Post-condition	La branche source et son historique restent inchangés ; la nouvelle branche conserve sa provenance et évolue indépendamment.

2.6.4 SC-04 — Télécharger après une correction sans changer de lien

Acteur principal	Visiteur anonyme qui utilise le lien de téléchargement.
Acteurs secondaires	Créateur du lien et rédacteur qui corrige le document.
Précondition	Un lien valide non épuisé existe ; le document est à l'état V1.
Scénario	1. Le rédacteur effectue une correction acceptée, qui produit l'état V2. 2. Le visiteur utilise le lien initial. 3. Le système autorise la demande et fournit un PDF cohérent correspondant à un état de référence qui inclut la correction. Variantes : une correction survient pendant la génération du PDF ; le fichier doit correspondre à un instant de référence défini. En cas d'échec du transfert, le traitement de la consommation dépend de la règle à valider. En parallèle, de nouvelles éditions et d'autres demandes de téléchargement peuvent avoir lieu.
Post-condition	Le visiteur reçoit le PDF actuel par le même lien ; la capacité est consommée selon une règle définie.

Traçabilité : EU-04, EU-06 ; EF-12 à EF-19 ; RM-01, RM-03.

2.6.5 SC-05 — Distribuer le dernier exemplaire sous accès concurrents

Acteur principal	Visiteur qui demande le dernier téléchargement disponible.
Acteurs secondaires	Autres visiteurs concurrents, créateur du lien, rédacteurs et visiteurs déjà en attente.
Précondition	Le lien possède un seul téléchargement restant.
Scénario	1. Plusieurs demandes arrivent simultanément ; au plus une consomme le dernier téléchargement. 2. Le lien devient épuisé ; les autres visiteurs en sont informés et le créateur reçoit un e-mail. Variantes : le visiteur autorisé abandonne, la génération du PDF échoue ou l'e-mail ne peut être remis ; les politiques correspondantes restent à définir. En parallèle, les rédacteurs poursuivent les corrections et des visiteurs peuvent déjà attendre.
Post-condition	Aucun dépassement n'a lieu ; les visiteurs présents sont informés de l'épuisement et la notification au créateur est déclenchée. La reprise d'une tentative échouée suit la règle de consommation retenue.

2.6.6 SC-06 — Épuiser le crédit depuis plusieurs appareils

Acteur principal	Rédacteur qui écrit depuis deux appareils.
Acteurs secondaires	Autres rédacteurs qui modifient les mêmes documents.

Précondition	Le même rédacteur est reconnu sur les deux appareils et dispose d'un faible crédit restant.
Scénario	1. Les deux appareils soumettent simultanément des modifications. 2. Le système n'accepte que les écritures compatibles avec le crédit restant ; le total quotidien ne dépasse pas 10 000 caractères. 3. Le crédit est rechargé au début de la période quotidienne suivante. Variantes : une opération dépasse le reliquat, une demande arrive au changement de journée ou une autre adresse est utilisée ; le traitement dépend des règles de comptage, de calendrier et d'identification. En parallèle, d'autres rédacteurs modifient les mêmes documents.
Post-condition	Le crédit n'est pas négatif, les écritures refusées ne sont pas annoncées comme acceptées et la recharge s'applique à la période suivante.

2.6.7 SC-07 — Détecter un abus et observer le blocage

Acteur principal	Rédacteur qui tente de dépasser les limites de façon répétée.
Acteurs secondaires	Représentant de l'association qui consulte l'état des personnes bloquées.
Précondition	Les limites sont atteintes et les règles de détection ainsi que la durée du blocage sont définies.
Scénario	1. Les tentatives répétées déclenchent la détection d'un abus. 2. Le système applique un blocage temporaire et l'association peut le voir dans l'état de l'application. 3. À l'expiration, le blocage cesse de s'appliquer. Variantes : le rédacteur utilise plusieurs appareils ou identités déclarées ; les garanties dépendent du modèle d'identification. En parallèle, les autres personnes poursuivent leur usage normal.
Post-condition	Le blocage a expiré ; les quotas continuent de s'appliquer indépendamment du blocage.

Traçabilité : EU-05, EU-07 ; EF-28, EF-29, EF-33.

2.6.8 SC-08 — Publier au pic et consulter l'état

Acteur principal	Visiteur qui utilise le lien pendant le pic de publication.
Acteurs secondaires	Association qui consulte l'état, 14 999 autres visiteurs et 300 rédacteurs actifs.
Précondition	Le lien est annoncé pour 500 exemplaires ; le document peut encore être corrigé et le système est déployé.

Scénario	1. 12 000 visiteurs arrivent au cours de la première minute tandis que les rédacteurs poursuivent leur travail. 2. Chaque visiteur reçoit un état : téléchargement immédiat, attente ou épuisement. 3. L'association consulte les documents ouverts, les liens et leurs capacités restantes, ainsi que les personnes bloquées. 4. Après la dernière consommation, les visiteurs encore présents sont informés de l'épuisement. Variantes : les arrivées sont plus concentrées que la moyenne, les visiteurs réessaient ou la génération ralentit ; le profil et les résultats doivent être consignés. En parallèle, l'historique, les notifications et la supervision restent sollicités.
Post-condition	Les limites de téléchargement, d'édition et de crédit sont respectées ; chaque visiteur reçoit une réponse selon les seuils fixés ; les traitements inutiles pour le lien épuisé cessent.

2.7 Vérification et traçabilité des contraintes

La vérification combine une argumentation sur les mécanismes choisis et des essais reproductibles. Les tests établissent un résultat dans les conditions exécutées ; ils ne démontrent pas seuls une garantie universelle. Un résultat identique sur tous les navigateurs ne suffit pas à établir la conservation de chaque modification acceptée.

Source	Exigences principales	Vérification prévue	Preuves attendues
C1	RM-01 ; EF-16 ; ENF-01	V-01 : capacité 1 puis 500 ; demandes concurrentes au-delà du plafond ; nouvelles tentatives.	Décisions, consommations et refus ; zéro dépassement ; tirage atteignable.
C2	RM-02 ; EF-05, EF-06 ; ENF-02, ENF-03	V-02 : éditions au même endroit, insertions, suppressions et remplacements ; arrêt et comparaison.	Opérations acceptées, oracle d'intégration et copies finales.
C3	RM-03 ; EF-14, EF-15 ; ENF-04	V-03 : lien créé avant correction ; usage après correction ; édition pendant export.	Révision de référence, PDF et comparaison du texte.
C4	RM-04 ; EF-23 à EF-27 ; ENF-05	V-04 : consommation concurrente sur plusieurs appareils puis bascule du jour.	Décompte par identité/journée, acceptations et refus.
C5	EF-17 à EF-21 ; ENF-07, ENF-08	V-05 : 15 000 visiteurs, 12 000 en première minute ; 300 rédacteurs actifs.	Profil d'arrivée, réponses réconciliées, latences, erreurs et invariants P0.
C6	EF-21 ; ENF-09, ENF-10	V-06 : épuisement avec visiteurs encore présents ou en attente.	Délais d'information et absence de nouvelle distribution.

Les fonctions restantes font l'objet de vérifications ciblées : création/partage (EF-01 à EF-04),

présence et historique (EF-07 à EF-11), notification (EF-22), abus et blocage (EF-28, EF-29), supervision (EF-30 à EF-33). Les ENF-11 à ENF-24 appellent des revues de sécurité, d'infrastructure et de livrables. La future matrice détaillée associera à chaque identifiant un test exécuté et une preuve conservée.

2.8 Questions ouvertes et objectifs à valider

Les points suivants n'empêchent pas l'extraction des besoins, mais empêchent de figer certaines conditions d'acceptation. Ils seront résolus par des décisions datées, liées aux exigences concernées.

ID	Question à trancher	Incidence
Q-01	Un téléchargement est-il consommé à la réservation, à la remise d'un accès au fichier, au début du transfert ou à sa réception complète ? Une nouvelle tentative compte-t-elle à nouveau ?	Plafond exact, abandons, stockage S3 et tests de distribution.
Q-02	À quel événement une modification est-elle « acceptée » ?	Preuve d'absence de perte et gestion des réponses réseau perdues.
Q-03	Quel instant précis représente T1 : arrivée de la demande, décision d'autorisation ou capture du contenu ?	PDF actuel et cohérent pendant l'édition.
Q-04	Quels caractères sont décomptés, avec quelle unité Unicode ? Quel coût retenir pour la copie initiale d'une nouvelle version ou branche ?	Crédit quotidien et équité du comptage ; les autres opérations ont des coûts proposés au chapitre 3.
Q-05	La journée UTC est proposée pour le quota du rédacteur : à confirmer. Elle reste distincte de la journée locale du propriétaire utilisée pour le versionnage.	Bascule de période et limite quotidienne de 10 000 caractères imposée par C4.
Q-06	Décision : nom et e-mail, code de vérification et identité métier stable entre sessions. Reste à définir la reprise d'accès et le traitement de plusieurs adresses d'une même personne.	Crédit commun entre appareils ; la possession d'une adresse ne prouve pas l'unicité d'une personne.
Q-07	Quels seuils, fenêtre temporelle, durée et périmètre déterminent l'abus et le blocage ?	EF-28 et EF-29 vérifiables.
Q-08	Décision : lecture sur la branche, écriture sur sa version active ouverte, restauration par le propriétaire vers une nouvelle branche ; les droits révoqués le restent. Restent à préciser la création des liens et l'accès à la supervision.	Contrôle d'accès côté serveur et matrice de droits au chapitre 3.
Q-09	Décision : révisions techniques dans une version métier, versions successives dans une branche, restauration indépendante. Restent à fixer la rétention et le coût de la copie initiale.	Provenance et clôture décrites au chapitre 3.
Q-10	« Version ouverte » désigne ici l'état éditable d'une branche. Que signifient encore « document ouvert » dans la supervision, « lien actif » et « personne présente » ?	Éviter de confondre état métier, présence en ligne et état de diffusion.

ID	Question à trancher	Incidence
Q-11	Quels seuils chiffrés pour « presque aussitôt », « rapidement » et le temps de réponse sous charge ?	Qualification des performances.
Q-12	Quel comportement lorsque le PDF ou l'e-mail échoue ? Quelle tolérance aux doublons d'e-mail ?	Cohérence entre consommation, fichier remis et notification.
Q-13	Quelle forme de pic à l'intérieur de la première minute et quelle durée d'observation après celle-ci ?	Reproductibilité des essais ; 200 arrivées/s est une moyenne, pas un maximum.
Q-14	Quels traitements cessent après épuisement et que deviennent les travaux déjà autorisés ?	Interprétation de C6 sans interrompre l'édition ni les autres liens.
Q-15	Quelles bonnes pratiques précises des autres cours et quel budget de travail faut-il retenir ?	Critères de revue et choix réalisables dans le plan AWS imposé.

Objectifs de mesure proposés, non validés. Pour rendre les termes qualitatifs vérifiables, une première discussion peut partir des valeurs ci-dessous. Ces valeurs ne proviennent pas du sujet et ne constituent pas encore des engagements.

Mesure	Candidat à valider	Conditions à fixer
Diffusion d'une modification	p95 $\leq$ 1 s	Départ à l'acceptation, fin à l'affichage distant ; sous charge définie.
Premier état métier d'une demande	p95 $\leq$ 2 s ; max. $\leq$ 10 s	Inclut les demandes mises en attente ; ne signifie pas livraison du PDF en 2 s.
Information d'épuisement	p95 $\leq$ 2 s	Depuis la dernière consommation ; fixer aussi un maximum pour tous les visiteurs actifs.
Convergence après arrêt des éditions	max. $\leq$ 5 s	Échanges disponibles, toutes les opérations acceptées identifiées.

Table 2.13 – Candidats de qualification à négocier ; aucun seuil chiffré de cette table n'est fourni par l'énoncé.

Le protocole de charge devra préciser la concentration des clics, la durée du scénario, le nombre d'opérations par rédacteur, leur taille, la taille des documents et leur répartition. Il devra mesurer des rédacteurs réellement actifs, et non seulement 300 connexions ouvertes.

2.9 Validation et gestion de la spécification

La revue suivra les critères du cours [2, diapositives 37–39] : **validité** des besoins, **consistance** entre quotas et concurrence, **complétude** des fonctions demandées, **réalisme** au regard du délai et du budget, **vérifiabilité**, **compréhensibilité**, **traçabilité** et **adaptabilité**.

Les points de tension principaux sont la fraîcheur du PDF pendant l'attente, la preuve de conservation des écritures admises lors d'une clôture et l'identité commune face aux quotas multiappareils. Les décisions prises sont détaillées au chapitre 3 ; leurs garanties techniques demandent encore des essais avant de définir l'API et les tests définitifs.

À chaque évolution, les identifiants restent stables ; une exigence modifiée référence la question résolue, sa source et les tests affectés. Les diagrammes PlantUML et le texte sont révisés

ensemble. La prochaine partie du cahier pourra alors justifier les choix d'architecture et leurs alternatives à partir de cette base.

Chapitre 3

Conception

3.1 Gestion du crédit, des débits et des écritures

Dans cette section, « facturation » désigne uniquement la consommation du crédit d'écriture, sans paiement. Le sujet impose un crédit de 10 000 caractères par rédacteur et par jour, commun à ses documents et à ses appareils, ainsi que l'impossibilité d'écrire au-delà de ce crédit et un blocage temporaire en cas d'acharnement [1]. Il ne définit ni l'unité Unicode, ni l'heure de recharge, ni le détail des opérations facturables. Les règles qui suivent constituent donc un *modèle proposé pour le MVP*, à confronter au fonctionnement réel de l'éditeur retenu et à valider avec le porteur du projet (Q-02, Q-04 à Q-07 et Q-09).

3.1.1 Objet et unité du crédit

Le crédit mesure le texte *inséré et accepté* par un rédacteur pendant une journée. Il ne mesure ni les touches pressées, ni les échanges avec le système, ni la taille finale du document, ni les modifications reçues des autres rédacteurs. Le coût est cumulé pour un même utilisateur, quels que soient ses documents, ses sessions et ses appareils.

L'unité proposée est le *point de code Unicode*, qui ne correspond pas toujours à un caractère perçu par le lecteur. Ainsi « Bonjour » contient sept points de code, « é » précomposé en contient un, tandis que « e » suivi d'un accent combinant en contient deux. Les espaces et les retours à la ligne saisis comptent ; les éléments ajoutés automatiquement pour le fonctionnement de l'éditeur ne comptent pas.

Le calcul proposé compte les points de code du texte effectivement inséré, sans transformer silencieusement sa forme Unicode. Deux suites visuellement identiques, comme les deux formes de « é » ci-dessus, peuvent donc avoir des coûts différents : ce choix doit être confirmé. Une règle commune doit aussi fixer comment compter les retours à la ligne, sans changer à l'insu des rédacteurs le texte ou les positions d'écriture.

3.1.2 Coût proposé des opérations

Opération	Coût proposé
Insertion ou collage	Nombre de points de code du nouveau texte inséré.
Suppression	Zéro, sans remboursement d'une insertion antérieure.
Remplacement	Nombre de points de code du texte nouvellement inséré.
Mise en forme seule, présence, déplacement du point d'insertion	Zéro.
Opération refusée ou retransmise après acceptation	Aucun débit définitif supplémentaire ; une retransmission doit retrouver le résultat de l'opération initiale.
Annulation	Zéro si elle supprime du texte ; coût des insertions si elle réinsère du texte.
Copie initiale d'une nouvelle version ou branche	Coût encore ouvert ; ne pas assimiler cette copie à une frappe du rédacteur.
Réinsertion volontaire d'un texte refusé	Coût des nouveaux points de code insérés, soumis aux droits et au crédit courants.

Ces coûts sont des propositions de conception, non des précisions énoncées par le sujet. Les remplacements et les annulations devront être vérifiés. Selon la décision du porteur, la restauration crée une branche indépendante : elle ne réécrit pas la version source. La gratuité de la copie initiale reste à valider ; une saisie ultérieure dans la nouvelle branche suit les règles ordinaires.

3.1.3 Acceptation, plafond et refus

Une frappe peut être préparée et affichée localement, puis reçue par le système, sans être encore acceptée. Le modèle distingue : *préparée localement*, *reçue*, *admise* pour traitement, *enregistrée* dans l'historique de l'éditeur, *acceptée* et *refusée*. L'admission ne vaut pas acceptation. Une confirmation d'acceptation ne doit être émise qu'après preuve que l'écriture durable et son débit sont cohérents. L'articulation exacte entre confirmation, retransmission et conservation des écritures reste à établir ; l'affichage local ne constitue pas cette preuve.

Soit $C(u, j)$ la consommation acceptée du rédacteur u pour la journée j. L'invariant visé est :

$$C(u, j) \leq 10\,000.$$

Une opération de coût c ne peut être acceptée que si $C(u, j) + c \leq 10\,000$. Le refus entier est proposé lorsque l'opération dépasse le reliquat : avec 30 points de code disponibles, un collage de 50 points de code est refusé sans débit définitif. L'interface devra expliquer le refus et conserver le texte non accepté afin que le rédacteur puisse le récupérer. Une opération déjà acceptée, identifiée de manière stable, ne doit pas être débitée une seconde fois lors d'une retransmission.

3.1.4 Journée, débit et abus

La journée civile UTC est proposée : elle commence à 00 h 00 UTC et se termine juste avant le minuit suivant, sans report du reliquat. L'horloge du serveur fait foi et une opération appartient au jour de son acceptation, non à l'heure annoncée par le navigateur. Une fenêtre glissante de 24

heures depuis la première écriture produirait une autre règle ; ce n'est pas le choix proposé ici. La bascule de jour pendant une opération en cours devra être testée.

Le *crédit* est une quantité cumulée par jour ; le *débit* est un nombre de caractères ou d'opérations par unité de temps ; la *protection contre les abus* constitue encore une autre règle. Le sujet n'impose pas d'étaler les 10 000 caractères sur la journée : un collage volumineux peut être légitime. Aucun plafond arbitraire de caractères par seconde n'est fixé. Les seuils, la fenêtre d'observation, la durée et le périmètre d'un blocage temporaire restent à valider (Q-07).

3.1.5 Identité, sessions et appareils

Le crédit appartient à un utilisateur identifié de manière stable. Un rédacteur peut avoir plusieurs sessions simultanées ; fermer le navigateur, se reconnecter, changer de version ou créer une branche ne renouvelle pas son crédit. Le modèle conceptuel minimal comprend un *rédacteur*, une *session* avec expiration, une *consommation journalière* et une *opération d'écriture* identifiée de façon stable avec son état. Le crédit reste conservé par le système, indépendamment du navigateur. Un éventuel repère de navigateur ne prouve pas l'existence d'un appareil physique distinct.

Le sujet demande le nom et l'adresse e-mail sans compte classique [1]. Le porteur ajoute une vérification par code envoyé à cette adresse avant d'établir l'identité métier stable [3]. Plusieurs sessions peuvent alors partager le même crédit. La possession d'une adresse ne prouve pas qu'une personne ne dispose pas d'autres adresses. Il faudra vérifier qu'un même rédacteur conserve un crédit commun lors de plusieurs connexions, même si l'éditeur lui attribue plusieurs identifiants internes. Le choix de l'outil de vérification reste ouvert ; aucun mot de passe ni dispositif de reconnaissance du matériel n'est imposé.

3.1.6 Cohérence du débit et décision à valider

Un compteur atomique évite certains dépassements concurrents, mais ne rend pas à lui seul atomique l'ensemble « consommation du crédit + conservation de l'écriture ». Les échecs possibles sont : crédit débité sans écriture conservée, écriture acceptée sans débit, double débit après retransmission et dépassement entre appareils ou documents. Une réservation, un journal d'opérations sans double effet et une reprise après incident sont des pistes à étudier, sans garantie annoncée tant que le point de validation des écritures et les scénarios de panne n'ont pas été vérifiés.

Le refus d'une opération doit intervenir avant qu'elle ne soit diffusée comme acceptée et ne doit pas perturber les opérations suivantes. Le mécanisme de validation et l'ordre des confirmations seront examinés sur la version intégrée de l'éditeur. Un débit de crédit suivi séparément d'une sauvegarde ne démontre ni l'invariant de crédit ni la conservation des écritures acceptées. La qualification demandera des essais concurrents, des retransmissions et des pannes à chaque frontière d'acceptation.

3.1.7 Diagramme de classes du crédit

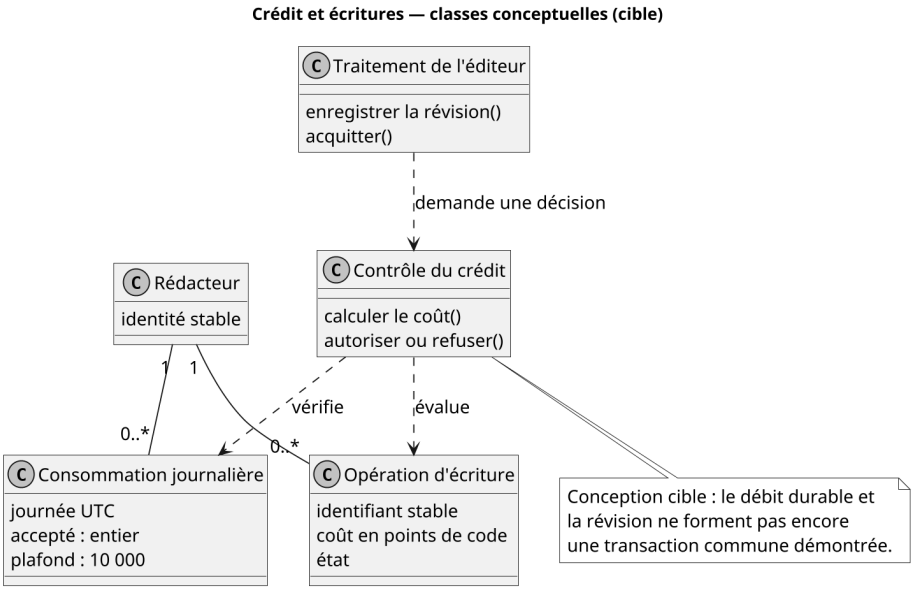


FIGURE 3.1 – Crédit d'écriture : objets conceptuels et frontière avec l'éditeur (cible).

Ce diagramme sépare le compteur journalier, l'opération identifiée et la révision de l'éditeur. Le service de crédit et le journal d'opérations sont une cible de conception, pas des classes déjà opérationnelles.

3.1.8 Diagramme d'activités du crédit

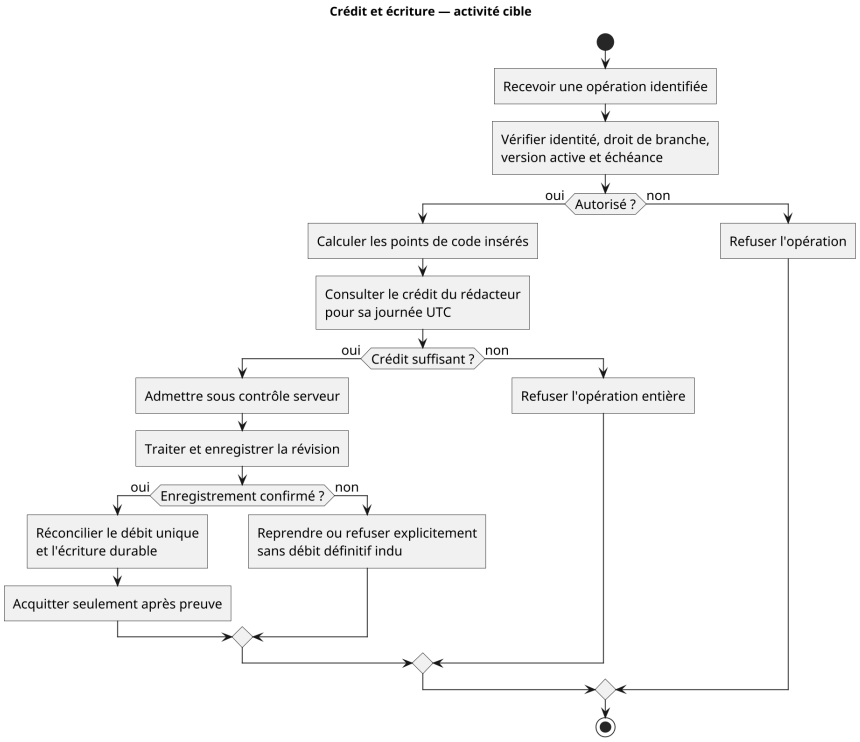


FIGURE 3.2 – Contrôle proposé d'une écriture, du refus à la confirmation.

Le parcours vérifie d'abord l'identité, le droit et la version ; il calcule ensuite le coût des seuls points de code insérés. Le débit durable, le traitement de l'échec et la confirmation demandent encore un protocole démontré.

3.1.9 Diagramme de séquence du crédit

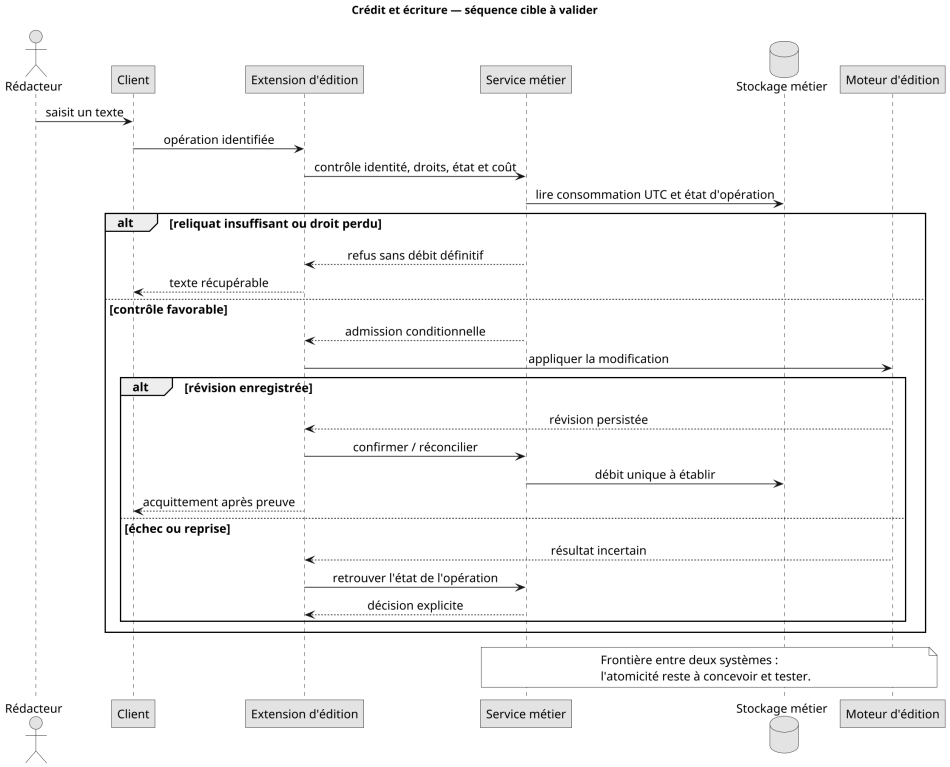


FIGURE 3.3 – Échanges visés pour écrire avec crédit suffisant ou refuser.

La frontière entre le stockage métier et les révisions de l'éditeur est volontairement visible. La séquence indique un besoin de réconciliation ; elle ne prétend pas qu'une transaction atomique existe déjà entre les deux systèmes.

3.2 Historique, versionnage et branches

Les règles de cette section précisent l'usage du journal décidé par le porteur du projet [3]. Elles complètent le sujet initial. Le code réalise les parcours locaux de branches, de versions et de clôture ; les garanties de panne et de charge restent à démontrer.

3.2.1 Termes et organisation des documents

Une *révision technique* est un état enregistré au fil des modifications de l'éditeur. Une *version métier* (V1, V2, etc.) est une étape de travail explicitement ouverte, puis figée. Elle peut contenir de nombreuses révisions techniques : chaque frappe ne crée pas une nouvelle version métier. Une *branche* rassemble une succession de versions métier et possède son propre historique. Une version appartient à une seule branche ; une branche ne possède au plus qu'une version ouverte à l'édition. Pendant une clôture, sa version active est en état FERMETURE et la suivante n'est pas encore éditable.

Le projet d'édition porte le propriétaire unique, le fuseau IANA et la prochaine échéance. Chaque branche porte ses droits et sa version active ; chaque version porte son numéro, son état, sa référence de pad et sa révision finale lorsqu'elle est figée. Une provenance désigne la branche, la version et la révision sources. Une clôture possède une cause, une échéance et un avancement. Ces

objets existent dans le schéma métier actuel, sous des noms anglais ; la table historique extttdocuments joue le rôle de projet d'édition. Elle sera renommée seulement par une migration préservant les données.

Par exemple, la branche A contient V1 figée puis V2 éditable. Si son propriétaire restaure une révision de V1, le système crée la branche B et sa première version éditable à partir de ce texte. La provenance de B mentionne A, V1 et la révision choisie. V1 et V2 de A ne changent pas ; les modifications ultérieures de A et de B ne se propagent pas entre les branches. Même avec un texte initial identique, les deux documents de travail sont distincts. Le MVP ne prévoit pas de fusion automatique.

3.2.2 Autorisations et édition en ligne

Rôle sur la branche	Lire ses versions et révisions	Écrire dans la version active	Clore, restaurer et attribuer les droits
Propriétaire	Oui.	Oui, tant qu'elle est OUVERTE.	Oui ; la restauration crée une nouvelle branche.
Collaborateur en écriture	Oui.	Oui, tant qu'elle est OUVERTE et que le droit subsiste.	Non.
Collaborateur en lecture	Oui.	Non.	Non.
Personne sans droit	Non ; un lien public de PDF suit une règle distincte.	Non.	Non.

Le droit de lecture porte sur la branche et son historique. Le droit d'écriture vise uniquement la version active ouverte ; une version figée reste en lecture seule. La bascule vers V2 conserve les droits de la branche, y compris leurs révocations, sans nouvelle invitation. Une restauration n'hérite pas automatiquement des droits de la source : le propriétaire attribue les accès à la branche créée. Le serveur doit vérifier ces règles avant d'admettre une écriture ; un verrouillage visuel seul ne suffit pas.

La saisie suppose une connexion opérationnelle au serveur d'édition. Lorsqu'une coupure est détectée, l'interface doit cesser d'accepter de nouvelles saisies. La détection n'est pas instantanée : le texte affiché localement pendant cet intervalle n'est pas pour autant accepté. À la reconnexion, le système doit vérifier de nouveau la version active, les droits et le crédit avant toute écriture ; il ne doit pas rejouer automatiquement une modification devenue interdite.

3.2.3 Fermeture manuelle et passage quotidien

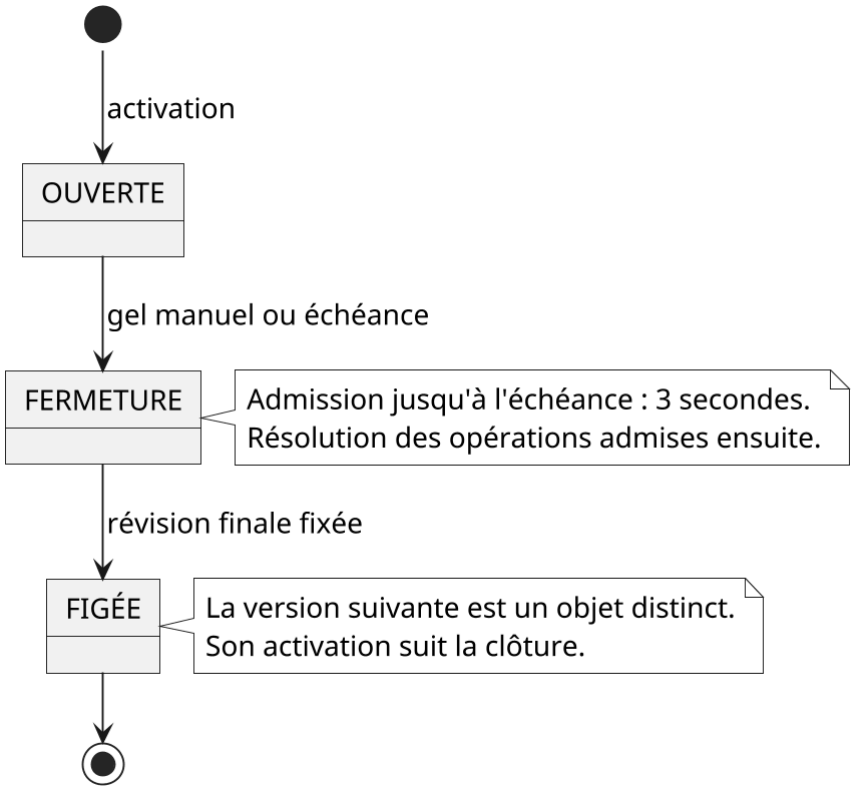


FIGURE 3.4 – Cycle métier d'une version ; la suivante est un autre document de travail.

Lors d'un gel manuel, le serveur enregistre la demande et fixe une échéance trois secondes plus tard selon son horloge. Il informe les clients, qui désactivent les nouvelles saisies. La marge permet l'admission d'opérations reçues par le serveur avant l'échéance, sous réserve des droits et du crédit ; une heure déclarée par le navigateur ne fait pas foi. Après l'échéance, toute opération encore non admise est refusée explicitement. Les trois secondes bornent la fenêtre d'admission, pas le temps total de persistance ou de création de V2.

La clôture suit un ordre : fermer l'admission, résoudre chacune des opérations admises, fixer la révision finale, figer V1, créer le nouveau document de travail depuis ce contenu, puis activer V2. Une opération admise peut encore être refusée avec une réponse explicite ; une opération déjà acceptée ne peut pas être abandonnée. V2 ne devient éditable qu'une fois la source finale déterminée. La reprise après incident doit reconnaître la clôture par son identifiant et ne créer qu'un seul successeur actif.

Le même mécanisme se déclenche à la fin de la journée civile du propriétaire, même s'il est absent. Son fuseau IANA est conservé explicitement ; le backend calcule et recherche périodiquement les échéances, sans minuteur permanent par document. Si la fermeture commence à minuit local, l'admission cesse trois secondes plus tard, puis la finalisation peut prendre davantage de temps : V1 n'est pas un instantané strict de minuit. Les droits de branche demeurent applicables à V2 lorsqu'elle est prête. Cette journée de versionnage, la journée UTC proposée pour le quota du rédacteur et la durée de ses sessions sont trois notions distinctes. La clôture ne recharge jamais le crédit.

3.2.4 Écritures refusées, invariants et décisions restantes

Une opération trop tardive est refusée avec une explication. Une proposition simple pour le MVP est un panneau ou un export du texte local, afin que le rédacteur puisse récupérer ce qu'il souhaite réinsérer lui-même dans V2. Cette réinsertion passe à nouveau par les droits, le crédit et l'acceptation habituelle. Le texte local complet peut déjà contenir des contributions acceptées : il ne faut pas le copier intégralement dans V2 comme s'il représentait seulement le refus. Le client local conserve maintenant cette copie dans le navigateur après rechargement ; les essais de reconnexion longue restent à faire.

Les invariants visés sont : au plus une version OUVERTE par branche ; aucune écriture ordinaire dans une version FIGÉE ; aucune perte d'une écriture acceptée pendant la clôture ; aucune nouvelle admission après l'échéance ; aucune activation du successeur tant que la révision finale reste indéterminée ; aucune propagation automatique entre branches ; aucune recharge de crédit par changement de version ; un seul successeur pour une même clôture, y compris après reprise.

Restent à valider la rétention des révisions, la création quotidienne d'une nouvelle version lorsque le texte n'a pas changé, la facturation de la copie initiale et la façon de distinguer précisément les opérations déjà en transit lorsque le gel commence. La preuve des garanties d'admission et de persistance dépendra de l'intégration réelle avec l'éditeur.

3.2.5 Diagramme de classes des versions

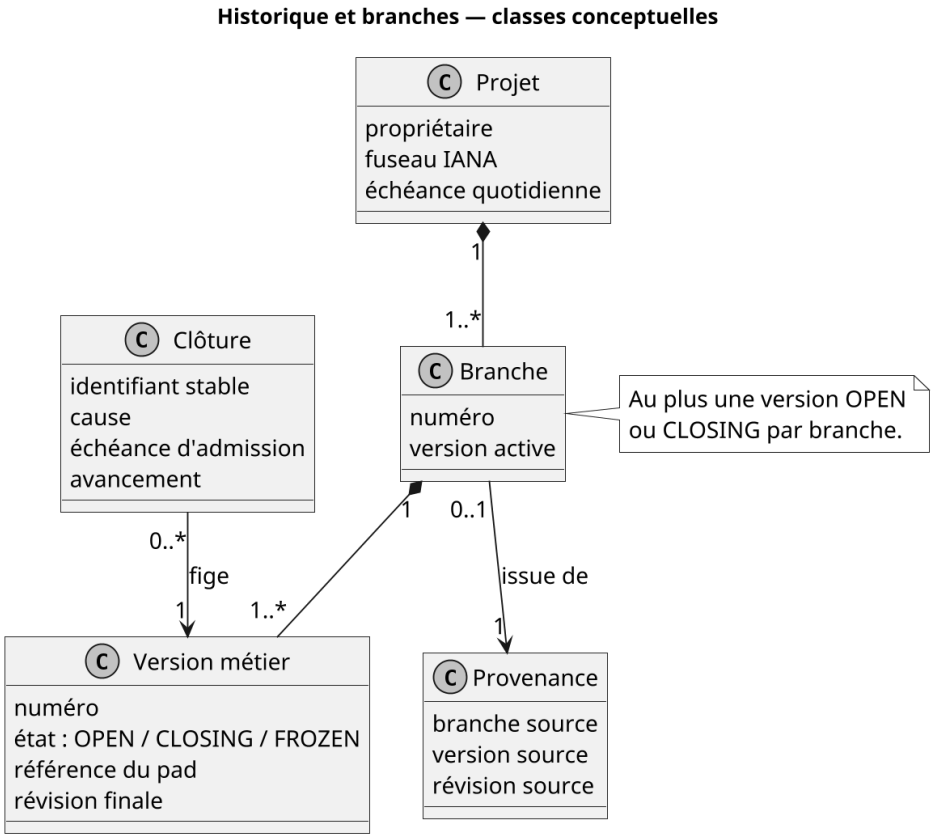


FIGURE 3.5 – Projet, branches, versions, provenance et clôture.

La propriété est portée par le projet. Une branche ne possède pas un propriétaire autonome ; elle conserve ses permissions au passage à V2. Chaque version métier référence un pad distinct et

conserve ses révisions techniques dans ce pad.

3.2.6 Diagramme d'activités de la clôture

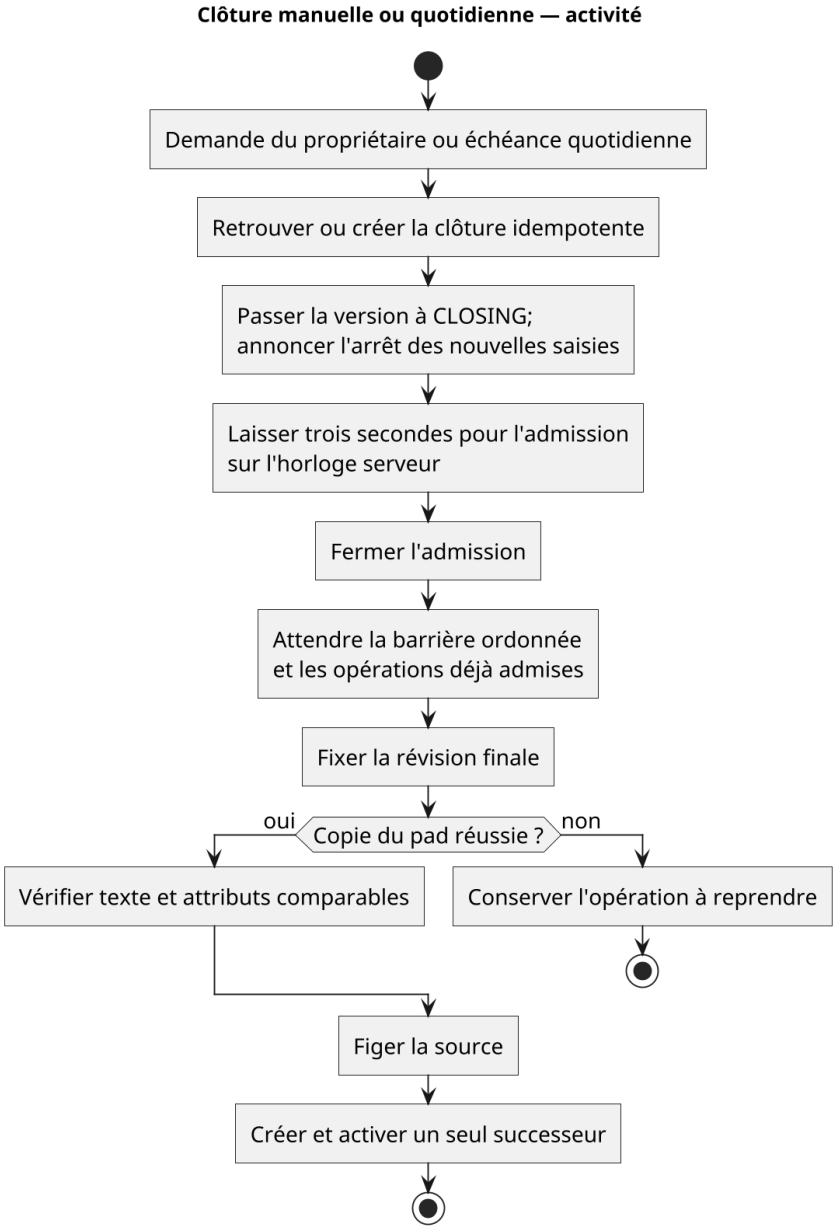


FIGURE 3.6 – Clôture ordonnée avec reprise après incident.

Les trois secondes bornent l'admission. La barrière fixe la révision finale avant la création du successeur ; une panne laisse une opération à reprendre et ne justifie pas l'activation prématurée de V2.

3.2.7 Diagramme de séquence de la clôture

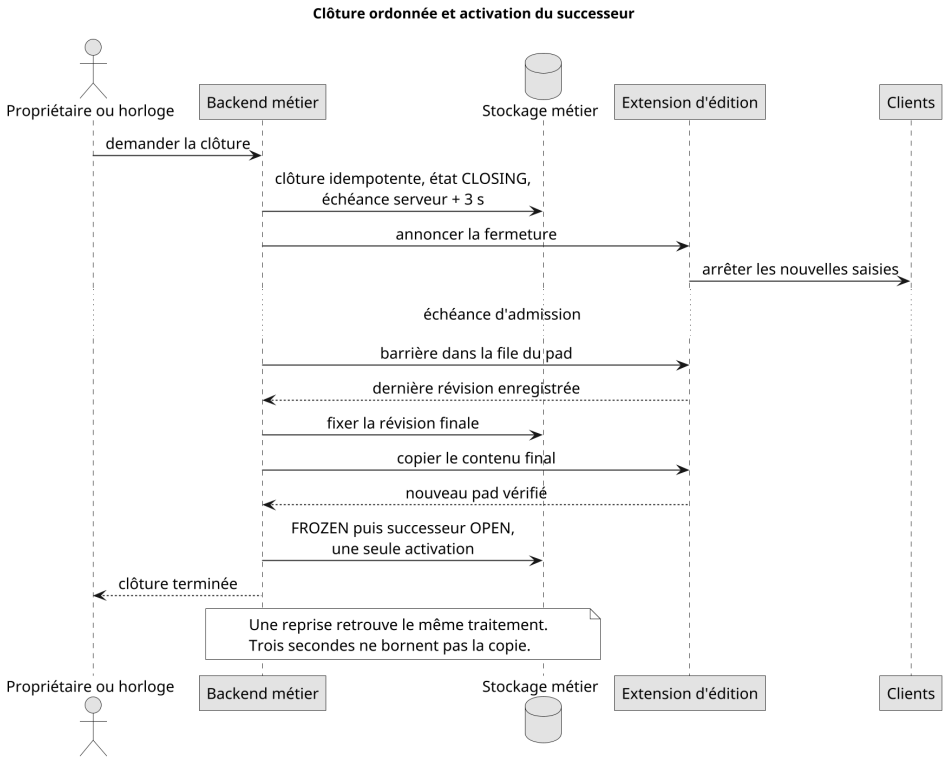


FIGURE 3.7 – Échanges entre propriétaire ou horloge, backend, éditeur et clients.

La restauration suit une autre demande propriétaire : elle copie une révision choisie dans une nouvelle branche du même projet. Les changements des deux branches restent indépendants et les collaborateurs ne sont pas invités implicitement.

3.3 Création et partage des projets d'édition

3.3.1 Règles et responsabilités

Le terme *projet d'édition* désigne l'ensemble appartenant à un seul propriétaire ; l'ancienne table nommée `documents` représente actuellement cet objet et ne crée pas une seconde entité concurrente. La création idempotente produit la branche initiale, V1 ouverte et un pad indépendant. Le propriétaire contrôle toutes les branches. Un collaborateur reçoit READ ou WRITE uniquement sur la branche invitée ; WRITE inclut READ. Aucun droit n'autorise l'écriture dans une version figée.

Le propriétaire enregistre une invitation pour une adresse, un droit et une branche. Le lien contient un jeton aléatoire dont seule l'empreinte figure dans l'invitation persistée. Le lien n'accorde rien : le destinataire doit se connecter, vérifier la même adresse par code, puis accepter explicitement. L'acceptation crée ou modifie la permission de branche. Les états de l'invitation (PENDING, ACCEPTED, REVOKED, EXPIRED) sont distincts de ceux de l'envoi (QUEUED, SENT, FAILED). Un renvoi change le jeton et invalide l'ancien lien. Une invitation acceptée ne peut pas restaurer un droit révoqué par son rejeu.

3.3.2 Diagramme de classes du partage

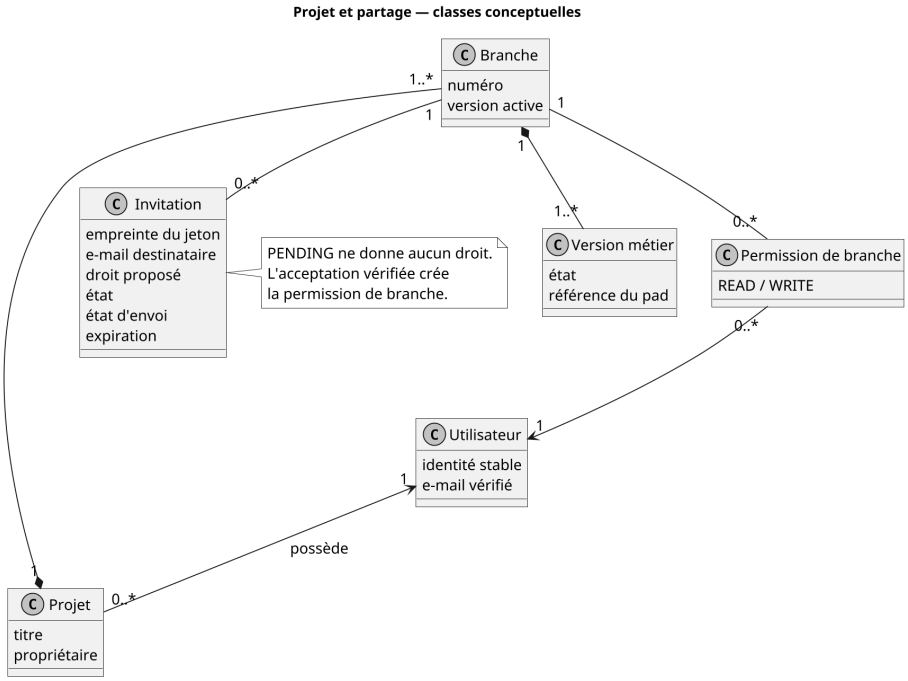


FIGURE 3.8 – Projet propriétaire, permission de branche et invitation distincte.

Les noms français désignent des concepts métier ; le code existant utilise notamment `BusinessService` et `InvitationService`. La relation entre utilisateur et projet porte la propriété unique. Une invitation en attente est séparée de la permission effectivement accordée.

3.3.3 Diagramme d'activités de l'invitation

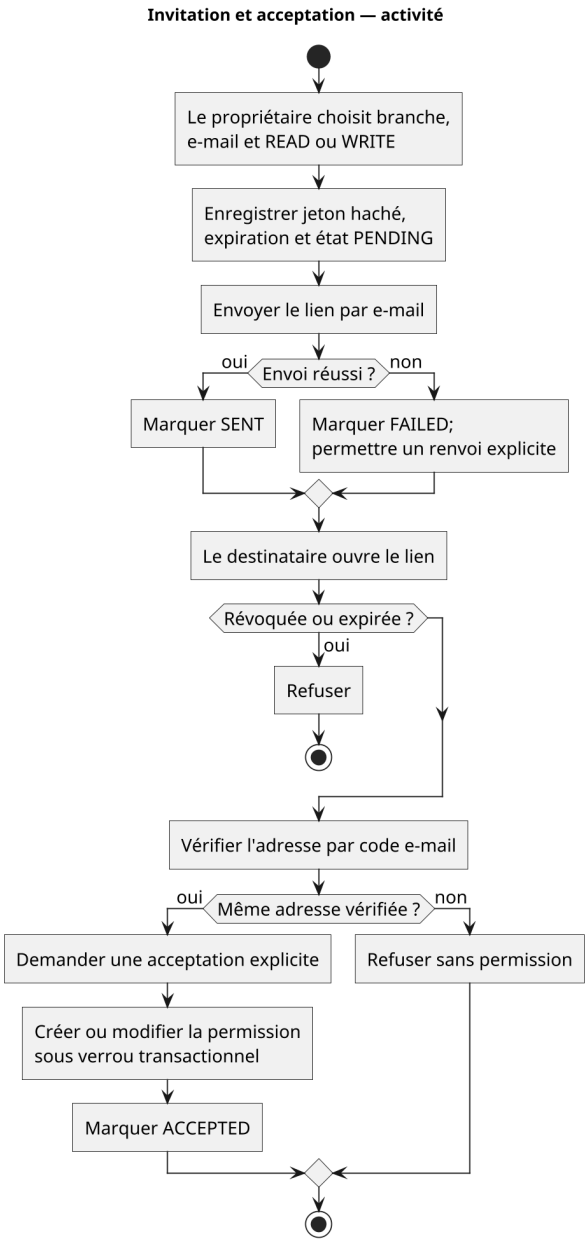


FIGURE 3.9 – De la création de l'invitation à l'acceptation ou au refus.

L'activité montre les refus dus à la mauvaise adresse, à l'expiration et à la révocation. Un échec d'envoi laisse l'état FAILED ; le propriétaire peut demander un renvoi qui renouvelle le lien. Le mode local garde les messages dans une boîte de test ; l'envoi SMTP externe n'est pas une livraison exactement une fois.

3.3.4 Diagramme de séquence du partage

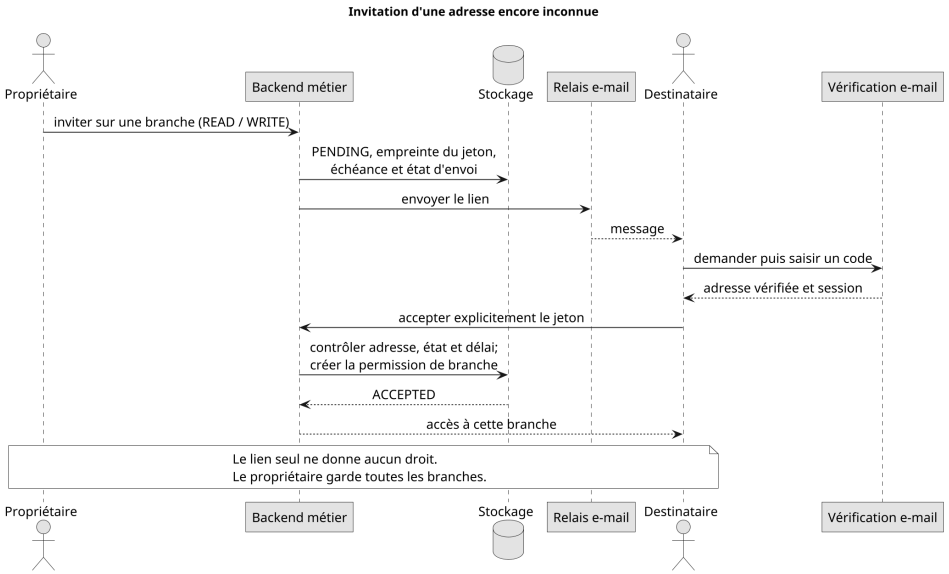


FIGURE 3.10 – Invitation d'une adresse encore inconnue et création du droit après vérification.

La création d'un compte avant l'invitation n'est pas nécessaire. Le service d'authentification prouve l'adresse ; le backend métier demeure responsable de la permission. L'acceptation sous verrou et l'unicité de la permission empêchent les doublons lors de requêtes concurrentes.

3.3.5 Invariants et limites

Une adresse invitée n'obtient aucune métadonnée d'une autre branche. Le propriétaire peut modifier ou révoquer les collaborateurs ; un collaborateur ne peut pas inviter. Les droits de branche survivent à V2 mais ne sont jamais copiés sur une branche restaurée. Le relais SMTP externe, les messages perdus entre enregistrement et envoi ainsi que les reprises après panne réclament encore une qualification. L'authentification déclarative historique reste uniquement un adaptateur de développement ; l'acceptation d'une invitation exige une session issue d'un code e-mail vérifié.

Chapitre 4

Implémentation locale et premiers résultats

4.1 Choix technologiques et compromis de réalisation

Le tableau couvre le parcours local et les besoins du projet encore à réaliser. « Coût » distingue licence, infrastructure, exploitation, développement et maintenance ; aucun tarif AWS n'est mesuré à cette étape. Les états indiquent ce que le code et les essais de cette livraison permettent réellement d'affirmer.

Table 4.1: Choix technologiques de l'application et compromis.

Besoin	Choix	Avantages ici	Limites	Coût et délai	État réel
Édition collaborative	Etherpad 3.3.6 [16].	Moteur de texte et révisions réutilisés.	Règles métier à intégrer ; capacité non mesurée.	Licence à vérifier ; hébergement et maintenance malgré le code ouvert.	Existant vérifié.
API métier	Node.js 20, TypeScript 5.9.3, Fastify 5.12.7 [32].	Une seule base de code serveur ; typage et routes légères.	Validation, reprises et charge CPU à surveiller.	Réduit l'intégration ; développement métier nécessaire.	Existant vérifié.
Données métier	PostgreSQL 15, migrations et contraintes [28].	Transactions et unicité durables pour droits et traitements.	Sauvegarde et exploitation ; aucune transaction commune avec Etherpad.	Stockage et sauvegardes selon l'hébergement.	Intégré, testé localement.
Projets et branches	Table historique `documents` comme projet, branches et permissions relationnelles.	Préserve les données et l'autorité unique du propriétaire.	Nom ancien à migrer plus tard ; courses à tester.	Pas de service neuf ; tests métier nécessaires.	Intégré.

Besoin	Choix	Avantages ici	Limites	Coût et délai	État réel
Versions métier	Un pad distinct par version.	Historique isolé ; pas de fusion à développer.	Multiplication des pads et copies ; capacité non démontrée.	Délai favorable, stockage croissant à mesurer.	Intégré, testé localement.
Initialisation	Copier contenu final et attributs utiles, sans ancien historique.	V2 démarre de V1 sans dupliquer ses révisions.	Fidélité de tous les attributs et coût d'E/S à qualifier.	Copie et stockage à mesurer.	Texte et HTML testés.
Restauration	Nouveau pad et nouvelle branche avec provenance.	Source intacte ; absence de fusion automatique.	Copie supplémentaire et droits à réattribuer.	Moins de logique qu'une fusion ; stockage accru.	Intégrée, testée.
Gel	Plugin client/serveur et patch ciblé d'Etherpad 3.3.6.	Admission et barrière dans la file du pad ; retour visuel.	Maintenance du patch et fautes en transit à qualifier.	Principal risque de tests et de maintenance.	Parcours local testé ; C2 ouverte.
Clôture quotidienne	Worker backend, échéances persistées et fuseau IANA.	Pas de minuteur par document ; reprise idempotente.	Courses multi-instance et pics de clôture à mesurer.	Pas de plateforme supplémentaire ; tests de panne requis.	Intégrée, partiellement testée.
Identité vérifiée	Better Auth 1.7.13 avec Email OTP et PostgreSQL [25, 26, 27].	Réutilise une solution maintenue ; sessions et preuve d'adresse.	Dépend du relais e-mail ; l'adaptateur ancien reste local.	Intégration et e-mails ; pas de tarif affirmé.	Intégrée et testée localement.
Invitations	Jeton haché, table métier et permission de branche.	Aucun service d'autorisation ajouté ; acceptation explicite.	Échec d'envoi, expiration et révocation à surveiller.	Faible infrastructure ; effort de sécurité et de tests.	Intégrées, testées localement.
E-mails	Nodemailer 10.2.1 et SMTP configurable ; boîte mémoire locale [29].	Aucun destinataire réel en test ; fournisseur interchangeable.	Boîte locale non durable ; SMTP peut retarder ou dupliquer.	Transport et exploitation variables selon fournisseur.	Local testé ; SMTP externe non testé.
Crédit d'écriture	Contrôle serveur commun aux sessions, à concevoir.	Navigateur non autoritaire.	Débit et révision dans deux systèmes ; atomicité à démontrer.	Effort de conception et d'essais élevé.	Prévu, non implémenté.
Récupération	Copie locale volontaire et téléchargement du texte.	Pas de fusion automatique.	Peut contenir tout le document ; confidentialité locale.	Faible coût serveur ; tests de reconnexion restants.	Parcours local vérifié.
Interface	Page HTML/JavaScript légère et éditeur Etherpad.	Un parcours minimal sans second framework.	Ergonomie et accessibilité à approfondir.	Délai réduit ; maintenance du client.	Intégrée, contrôle visuel.

Besoin	Choix	Avantages ici	Limites	Coût et délai	État réel
Déploiement local	Compose, PostgreSQL, backend, Etherpad et passerelle.	Démarrage reproductible.	Ne prouve pas la haute disponibilité.	Ressources de la machine ; exploitation cible à définir.	Vérifié localement.
Validation et observabilité	Tests Node, PostgreSQL et Etherpad ; journaux existants.	Éprouve des courses et les droits réels.	Fautes, charge et métriques non couvertes.	Temps d'exécution et maintenance des tests.	Partiel.
Sauvegarde et AWS	Sauvegarde, supervision et services AWS à déterminer.	Respect possible de la région et de S3.	Aucune architecture ou restauration testée.	Coût de stockage et d'exploitation à estimer.	Prévu.
Rapport	LaTeX et PlantUML.	Sources modifiables et génération reproductible.	Dépendances et inspection visuelle nécessaires.	Temps de compilation et relecture.	Existant vérifié.

Un pad par version simplifie l'intégration à Etherpad dans ce périmètre sans fusion, mais ne constitue aucune preuve de capacité. Il faudra mesurer le nombre de pads actifs, les révisions, le stockage et les clôtures simultanées. Git n'est pas introduit : le besoin n'inclut ni fusion ni rebase, et sa synchronisation avec les pads ajouterait un second état à maintenir. Il ne remplacerait ni les droits ni la barrière de gel ; cette décision ne suppose aucune infériorité intrinsèque de performance de Git. Réutiliser la stack limite les nouvelles intégrations, l'apprentissage et l'exploitation, sans rendre gratuits les licences, l'hébergement, les sauvegardes ou la maintenance.

4.2 Organisation et interfaces de cette étape

Le code contient maintenant un backend Fastify, des migrations pour le schéma métier `business` de PostgreSQL, un plugin `ep_business` et une image Etherpad 3.3.6 construite localement. Les anciennes données de pads demeurent intactes, mais le nouveau parcours métier ne donne pas accès aux pads sans version enregistrée. Les sessions historiques de développement utilisent un cookie opaque et un hachage persistant avec adresse déclarée. Leur création reste limitée au mode local. Le nouveau parcours vérifie l'adresse par code e-mail avec Better Auth ; une session métier vérifiée conserve l'identifiant utilisateur stable, y compris pour une adresse déjà connue. Les autorisations restent dans le backend.

`POST /projects` crée le projet d'édition (représenté par la table historique extttdocuments), sa première branche, sa version ouverte et son pad. Les routes de lecture exposent les branches, les versions et le contenu d'une révision autorisée. Le propriétaire vérifié peut inviter une adresse encore inconnue, renvoyer ou annuler le lien, puis modifier ou révoquer un droit après acceptation. Une invitation en attente ne donne aucun accès. Le jeton n'est conservé dans la table que sous forme d'empreinte ; le fragment du lien n'est pas envoyé dans l'URL des requêtes HTTP. `POST /branches/:id/closures` et `POST /versions/:id/restorations` retournent un identifiant de traitement et le statut HTTP 202 ; les routes de suivi distinguent traitement en cours et résultat terminé. Une clé d'idempotence est exigée pour ces créations. Le contrat d'API complet reste à rédiger.

La passerelle locale réserve les API administratives Etherpad aux composants internes. Le plugin

consulte le backend pour les connexions et les messages d'écriture ; connaître l'identifiant du pad ne suffit pas pour y accéder par le navigateur. Le code et les commandes de construction sont décrits dans `application/README.md`. Les visiteurs de PDF n'utilisent pas l'éditeur.

4.2.1 Points techniques à vérifier pour le crédit

Le modèle du chapitre 3 compte les points de code Unicode. Il faut distinguer, dans l'implémentation, les octets, les unités UTF-16, les points de code et les graphèmes visibles : `String.length` compte des unités UTF-16 et ne mesure donc pas directement le crédit. Le décodage des changements d'Etherpad devra isoler le texte réellement inséré des éléments techniques. Il faudra aussi vérifier toute conversion des fins de ligne CRLF/LF avant le calcul, sans changer silencieusement le texte ou les positions d'écriture.

La correspondance entre l'identifiant stable de l'utilisateur (`user_id`), ses sessions et l'identifiant d'auteur d'Etherpad (`authorId`) reste à établir pour plusieurs connexions. Les hooks d'Etherpad, la persistance et l'ordre des acquittements devront être étudiés avant de placer le refus et la confirmation d'une écriture [19]. PostgreSQL, Valkey et `rate-limiter-flexible` sont des candidats pour certaines règles de crédit ou de protection contre les abus, sans assemblage validé. Un appel `consume()` suivi d'une sauvegarde ne résout pas l'atomicité entre débit et écriture.

4.3 Correspondance entre conception et code

Une branche et ses permissions sont des données métier. Chaque version utilise un pad Etherpad distinct ; ses révisions techniques restent dans ce pad. Le passage à V2 copie le contenu final de V1 sans recopier son historique. Pour restaurer une révision antérieure, le service copie d'abord le pad source avec son historique vers un pad temporaire, applique `restoreRevision` à cette *copie*, puis crée le pad définitif de la nouvelle branche. Le pad source reste intact. La comparaison du texte et du HTML a été testée sur une révision avec gras et italique ; elle ne prouve pas encore la fidélité de tous les attributs possibles.

Table 4.2: Mécanismes implémentés et preuves restantes.

Mécanisme	Support Etherpad 3.3.6	Adaptation locale	État	Preuve restante
Branches et versions	Pads et révisions [18].	Tables métier, contraintes d'unicité, copie du pad final.	Intégré ; essai V1 vers V2 réussi.	Reprises aux pannes injectées.
Historique et restauration	Lecture de révision ; `restoreRevision` écrit dans le pad appelé [22].	Copie temporaire, restauration sur cette copie, pad indépendant et provenance.	Essai réel avec HTML réussi.	Attributs complexes et gros historique.
Permissions	Connexion et messages contrôlables par plugin [19].	Session métier, droit de branche, état de version et passerelle API.	Accès et révocation vérifiés localement.	Session longue et déploiement public.
Clôture	File par pad et acquittement après révision [20].	Patch versionné : admission et barrière dans la file ; délai de trois secondes.	Passage à V2 et annonce client observés.	Écritures en vol, pannes et concurrence de plusieurs instances.

Mécanisme	Support Etherpad 3.3.6	Adaptation locale	État	Preuve restante
Échéance quotidienne	Aucune règle métier native.	Worker central et calcul par fuseau IANA.	Test réel d'échéance dépassée ; tests de changements d'heure.	Arrêt/reprise pendant une clôture.
Crédit et invitations	Aucun couplage transactionnel avec le quota.	OTP et invitations intégrés ; autorisation de crédit à compléter.	OTP testé ; quota absent.	C4 et atomicité.

Une modification *reçue* arrive au serveur. Elle est *admise* si la décision prise dans la file du pad, avec l'horloge du serveur, autorise encore l'écriture. Elle est *enregistrée* quand Etherpad ajoute sa révision, puis *acquittée* quand le client reçoit `ACCEPT_COMMIT`. Le patch ajoute la décision et la barrière dans cette file ; un simple hook `handleMessage` avant la file n'aurait pas suffi. Les trois secondes bornent l'admission, non l'achèvement de la copie ou de la transaction métier.

Le client désactive l'édition après annonce de fermeture et garde une copie du texte local. Après rechargement, cette copie demeure consultable dans le navigateur. Elle peut contenir des contributions déjà enregistrées ; elle n'est pas le delta exact refusé et n'est pas réinsérée automatiquement. Une coupure n'est pas détectée instantanément, et un test de reconnexion après longue coupure reste nécessaire. Les droits et l'état de la version sont revérifiés côté serveur au prochain message. Le débit du crédit et l'enregistrement de la révision ne forment toujours pas une transaction commune.

4.4 Déploiement local et vérifications

L'environnement local comprend PostgreSQL 15, Etherpad 3.3.6, le backend et une passerelle Nginx. Le script de préparation génère des secrets hors Git ; les migrations précèdent le démarrage du backend. Les commandes figurent dans `application/README.md`. Lors de cette livraison, la compilation TypeScript, les tests de calendrier et les tests d'intégration contre les conteneurs ont été exécutés ; les résultats précis et leur date figurent dans `VALIDATION.md`. Les validations antérieures y restent distinguées.

4.5 Limites et infrastructure cible

Les branches, versions, permissions, clôtures, restaurations, sessions vérifiées et invitations e-mail sont codées ; leur qualification reste partielle. Le contrôle du crédit, les liens limités, les PDF, les notifications du dernier téléchargement et la supervision ne sont pas implémentés. La copie initiale d'une version n'est pas facturée dans ce code, sans fixer encore la règle métier définitive. Aucun essai de 300 rédacteurs ou de 15 000 visiteurs n'a été exécuté. La garantie C2 exige encore des essais sur les écritures simultanées, en transit et lors des pannes ; C4 n'est pas satisfaite.

La cible demeure le compte AWS du porteur, dans une région, avec les PDF conservés dans S3, sans clés AWS dans le dépôt et avec des permissions minimales [1]. Aucun service AWS supplémentaire ni coût n'est présenté comme validé ; aucune ressource AWS n'est déployée ici.

Bibliographie

[1] *Projet d'évaluation : Un éditeur de documents partagé. Développement web et cloud.* Sujet fourni par le porteur du projet, 3 pages. Fichier local : `sujet_projet_stage-1.pdf`.

[2] *Ingénierie des exigences. Lesson 2 – Spécification.* Support fourni, 39 diapositives, fondé sur Ian Sommerville, *Software Engineering*, 9e édition. Fichier source : `Lecon-2_GL-Spécification.pdf`. Les numéros cités désignent les diapositives.

[3] Porteur du projet. *Cadrage de l'historique, des versions métier, des branches, des droits, du gel quotidien et de la vérification e-mail.* Décisions communiquées le 25 octobre 2026. Ces précisions et extensions ne sont pas attribuées au sujet initial.

[4] John Day-Richter, Google. *What's different about the new Google Docs : Conflict resolution.* 22 septembre 2010. Source historique.

`https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_22.html`

[5] John Day-Richter, Google. *What's different about the new Google Docs : Making collaboration fast.* 23 septembre 2010. Source historique.

`https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs.html`

[6] Google, Aide Éditeurs Google Docs. *Partager des fichiers via Google Drive.* Fonctions et limites de partage.

`https://support.google.com/docs/answer/2494822?hl=fr`

[7] Google, Aide Éditeurs Google Docs. *Identifier les modifications apportées à un fichier.*

`https://support.google.com/docs/answer/190843?hl=fr`

[8] Google, Aide Éditeurs Google Docs. *Créer, afficher ou télécharger un fichier.*

`https://support.google.com/docs/answer/49114?hl=fr`

[9] Overleaf. *Collaborating in Overleaf.* Documentation du mécanisme OT et de WebSocket.

`https://docs.overleaf.com/collaborating/collaborating-in-overleaf`

[10] Overleaf. Dépôt officiel `overleaf/overleaf`, manifeste racine `package.json`, branche `main`, état consulté le 24 octobre 2026. Identifiant Git du blob observé : `8965f9ccbd7d772f1f32c33e1e013406ba238473`. Les dépendances peuvent évoluer.

`https://github.com/overleaf/overleaf/blob/main/package.json`

[11] Overleaf. *Microservices.* Documentation des éditions sur site, section architecture et processus d'édition/compilation.

`https://docs.overleaf.com/on-premises/getting-started/microservices`

[12] Overleaf. *Server Pro infrastructure.* Documentation de déploiement sur site.

`https://docs.overleaf.com/on-premises/getting-started/server-pro-infrastructure`

[13] Overleaf. *Data and backups.* Documentation de persistance et sauvegarde sur site.

`https://docs.overleaf.com/on-premises/maintenance/data-and-backups`

[14] Yjs. *Introduction.* Documentation officielle du modèle CRDT et de ses intégrations.

`https://docs.yjs.dev/`

[15] Yjs. *Document Updates.* Documentation officielle des propriétés et de l'API des mises à jour.

`https://docs.yjs.dev/api/document-updates`

[16] Etherpad. *Version 3.3.6.* Publication officielle, 21 septembre 2026.

`https://github.com/ether/etherpad/releases/tag/v3.3.6`

[17] Etherpad. *Docker.* Documentation correspondant au tag v3.3.6 : image, configuration et persistance.

`https://github.com/ether/etherpad/blob/v3.3.6/doc/docker.md`

[18] Etherpad. *HTTP API.* Documentation du tag v3.3.6, API 1.3.1.

`https://github.com/ether/etherpad/blob/v3.3.6/doc/api/http_api.md`

[19] Etherpad. *Server-side hooks.* Référence des points d'extension, à confronter au code de la version intégrée.

`https://docs.etherpad.org/api/hooks_server-side.html`

[20] Etherpad. *PadMessageHandler.ts*, tag v3.3.6. Traitement des messages, file par pad et acquittement des écritures. Code source consulté le 25 octobre 2026.

`https://github.com/ether/etherpad/blob/v3.3.6/src/node/handler/PadMessageHandler.ts`

[21] Etherpad. *Pad.ts*, tag v3.3.6. Ajout de révision et effets du hook de mise à jour. Code source consulté le 25 octobre 2026.

`https://github.com/ether/etherpad/blob/v3.3.6/src/node/db/Pad.ts`

[22] Etherpad. *API.ts*, tag v3.3.6. Implémentation de `getText`, `createPad` et `restoreRevision`. Code source consulté le 25 octobre 2026.

`https://github.com/ether/etherpad/blob/v3.3.6/src/node/db/API.ts`

[23] Etherpad. *collab_client.ts* et *pad.ts*, tag v3.3.6. Coupure, reconnexion et modifications locales en attente. Code source consulté le 25 octobre 2026.

`https://github.com/ether/etherpad/blob/v3.3.6/src/static/js/collab_client.ts`

`https://github.com/ether/etherpad/blob/v3.3.6/src/static/js/pad.ts`

[24] Etherpad. *ace2_inner.ts*, tag v3.3.6. Export du texte local complet. Code source consulté le 25 octobre 2026.

`https://github.com/ether/etherpad/blob/v3.3.6/src/static/js/ace2_inner.ts`

[25] Better Auth. *Email OTP.* Documentation officielle du plugin intégré pour la connexion par code et la vérification de l'adresse e-mail, consultée le 25 octobre 2026.

`https://better-auth.com/docs/plugins/email-otp`

[26] Better Auth. *Fastify Integration Guide.* Handler et lecture des sessions, consulté le 25 octobre 2026.

`https://better-auth.com/docs/integrations/fastify`

[27] Better Auth. *PostgreSQL Adapter.* Configuration de la base et schéma dédié, consulté le 25 octobre 2026.

`https://better-auth.com/docs/adapters/postgresql`

[28] PostgreSQL Global Development Group. *Documentation PostgreSQL 15.* Transactions et contraintes.

`https://www.postgresql.org/docs/15/`

[29] Nodemailer. *Nodemailer documentation.* Transport SMTP configurable.

`https://nodemailer.com/smtp`

[30] Etherpad. *ep_cursortrace.* Dépôt et instructions du plugin de positions d'écriture.

`https://github.com/ether/ep_cursortrace`

[31] npm. *ep_cursortrace 3.1.65.* Métadonnées de version et de moteur Node.js consultées le 25 octobre 2026.

`https://www.npmjs.com/package/ep_cursortrace`

[32] Fastify. *Migration Guide V5* et publication v5.12.7. Node.js 20 ou supérieur est pris en charge par la branche 5.

`https://fastify.dev/docs/v5.10.x/Guides/Migration-Guide-V5/`

`https://github.com/fastify/fastify/releases/tag/v5.12.7`

[33] ONLYOFFICE. *Document Server.* Dépôt officiel et description des fonctions d'édition de documents bureautiques.

`https://github.com/ONLYOFFICE/DocumentServer`